COMPLETE CERTIFICATION GUIDE

Databricks Certified Data Engineer Associate

Everything you need to pass — all seven exam sections, deep.

Exam version: May 4, 2026 onward

---

45 scored questions · 90 minutes · multiple choice · ~70% to pass
7 sections · PySpark & SQL · Lakeflow · Unity Catalog · Delta Lake
Master study guide with worked code, analogies, traps & practice questions

How to Use This Guide

Read this page first — it tells you where the points are and how to study

Before we start. *This guide has jokes in it. Not because Databricks certification is a laugh riot, but because you will remember the thing you smiled at, and forget the fourteenth grey bullet point in a row. If a "Coffee break" box makes a concept stick, it earned its place. Onward.*

The Exam Blueprint (memorize the weights)

The exam is 45 scored multiple-choice questions in 90 minutes. Sections are **not** equally weighted — spend your study time proportionally. Transformation (3), Ingestion (2), and Jobs (4) together are ~59% of the exam.

#	Section	Weight	≈ Questions	Core focus
1	Databricks Intelligence Platform	6%	~3	Architecture, Delta Lake, Unity Catalog, compute & cost
2	Data Ingestion and Loading	21%	~9	Auto Loader, COPY INTO, Lakeflow Connect, batch/stream/incremental
3	Data Transformation and Modeling	22%	~10	PySpark cleaning, joins, dedup, aggregation, tuning, gold objects, quality
4	Working with Lakeflow Jobs	16%	~7	Tasks, DAG, control flow, schedules & triggers
5	Implementing CI/CD	10%	~5	Git Folders, Automation Bundles, Databricks CLI
6	Troubleshooting, Monitoring & Optimization	10%	~5	Run history, Spark UI, Liquid Clustering, cluster failures
7	Governance and Security	15%	~7	Managed/external tables, GRANT/REVOKE/DENY, masking/RLS, ABAC

Your priority order (from your latest practice result): you scored strong on Domains 1, 2, and 4 — keep them warm but don't over-study them. Spend most of your remaining time on your weak spots, in this order: Domain 3 (Transformation) first — it's your lowest score and the single biggest domain (22%); then Domain 7 (Governance) and Domain 5 (CI/CD); then Domain 6 (Troubleshooting). The focus panel below distills the exact decision rules those four domains test.

Your Focus Panel — Rapid Recall for Your 4 Weak Domains

These are the highest-yield "when you see this, answer that" rules for the domains you're currently losing points on. Read this page daily until every line feels obvious.

Domain 3 — Transformation (your #1 priority)

When the question says...	Answer
Keep the latest row per key, with all columns, deterministically	Window + `row_number()` ordered desc, keep `rn==1` (not dropDuplicates, not groupBy+max)
Method vs function	

When the question says...	Answer
	Reshapes the table → `df.method()` (`withColumn`, `distinct`). Computes a column → `F.function()`. No `addColumn`, no `F.distinct`
PySpark `union()` and duplicates	Keeps dups (= `UNION ALL`); add `.distinct()`; matches by position (use `unionByName`)
Join a huge table to a tiny one	`broadcast()` the small side; Spark measures uncompressed in-memory size, not file size
Array → one row per element, keep empty arrays	`explode_outer` (plain explode drops them)
Fast distinct on billions, small error OK	`approx_count_distinct`; `count("*")` = all rows vs `count(col)` = non-null
Data quality: standard Delta table vs DLT pipeline	Standard table → `CHECK` constraint (rejects writes). DLT/pipeline → `CONSTRAINT … EXPECT` (CHECK not supported in DLT); bare EXPECT = monitor-only
Gold BI aggregation/join, or a source with updates/deletes	Materialized view (incremental refresh; streaming tables are append-only & don't recompute joins; plain view recomputes every query)
Extract a field from a JSON string / Variant column (SQL)	Colon operator `col:field.sub` (or `get_json_object`) — no `extract_json()`; SQL streaming reads use `STREAM()`, not `read_stream()`
Safe-to-re-run daily load	`MERGE` on the business key (idempotent); plain append duplicates

Domain 7 — Governance

When the question says...	Answer
Dropped a table and the files were deleted	It was a managed table (external keeps the files)
Has SELECT but still "permission denied"	Missing `USE CATALOG` + `USE SCHEMA` (separate privileges; SELECT never grants them)
REVOKE from a user who's in a granted group	Still has access via the group — use DENY to block (DENY beats GRANT)
Hide whole rows by who you are	Row filter; hide a field's value = column mask
Enforce masking across thousands of tables + future ones	ABAC policy + governed tags
Create an external table on an S3 path	Needs an external location + storage credential
Let a principal upload files to a volume	`WRITE VOLUME` (`READ VOLUME` to read); `INSERT` / `SELECT` are for table rows, not volumes

Domain 5 — CI/CD

When the question says...	Answer
Preview changes without deploying	`databricks bundle plan` (no `bundle show` / `describe`)
Human-readable overview of the bundle	`databricks bundle summary`
Check config before deploy (CI gate)	`databricks bundle validate`
Override a variable at deploy time	`--var="key=value"` (double-dash, equals, quotes; not `-v`, not `:`)
Variable holding a map/object	`type: complex` (not `dict`)
Same code to dev/test/prod	One bundle, swap with `-t <target>`; `mode: development` prefixes names + pauses schedules

When the question says...	Answer
Different node type / cluster size per environment	targets block overrides (e.g. `node_type_id`) in one `databricks.yml` — not a global cluster policy
Git op NOT for beginners / view commit history	`rebase` is discouraged (rewrites history); history = Git dialog History tab
What `bundle destroy` deletes	Removes deployed workspace resources, not your local files; commit & push then open a PR

Domain 6 — Troubleshooting

When the question says...	Answer
One task ⩾ others, Max shuffle read ⩾ median	Data skew → enable AQE skew join handling (not bigger cluster, not fewer partitions)
OOM during `collect()` / `toPandas()`	Driver memory (executor OOM = mid-shuffle/skew)
Spill (disk) but job finished	Memory overflowed to disk — slow, not a failure (OOM = crash)
Job slowly getting slower over weeks	Lakeflow Jobs run history vs baseline
Which upstream task blocks the rest	The DAG graph — find the earliest failed node
Cluster won't start (capacity/quota)	Cloud-side quota/IAM — check the event log, not your code
Auto-optimize tables without maintenance jobs	Predictive optimization; change layout keys later → Liquid Clustering
What compute runs predictive optimization / OOM in mapPartitions	PO runs on Databricks-managed serverless; `list(iterator)` in mapPartitions = executor OOM (keep it lazy, not a shuffle fix)

How This Guide Is Structured

Each section mirrors the official exam objectives and uses a consistent toolkit so you learn the same way every time:

Element	What it gives you
Easy picture	A plain-language analogy so the concept sticks
Exam tip	The exact phrasing/trigger words the exam uses and the expected answer
Trap	A common wrong answer or subtle gotcha to avoid
The contrast people get wrong	Two similar tools side by side, with the rule for choosing
One-Minute Recap	A dense summary box covering the whole section — review these last

Exam-Day Mechanics

Fact	Detail
Format	45 scored + some unscored (unmarked) items; multiple choice
Time	90 minutes (~2 min/question — comfortable; don't rush)

Fact	Detail
Code language	SQL when possible, otherwise Python (PySpark)
Aids	None allowed; proctored (online or test center)
Fee / validity	USD $200; certification valid 2 years
Passing	Databricks doesn't publish an exact cut score; ~70% is the working target

Test-taking strategy: read the last line of each question first — it tells you what's actually being asked (cheapest cost? force a shuffle? keep all columns?). Eliminate obviously wrong options, then choose between the two that remain using the "contrast" rules in this guide. Flag-and-return on anything that takes over 90 seconds. There's no penalty for guessing — never leave a question blank.

Blueprint: 1 Platform 6% · 2 Ingestion 21% · 3 Transformation 22% · 4 Lakeflow Jobs 16% · 5 CI/CD 10% · 6 Troubleshooting 10% · 7 Governance 15%. 45 Q / 90 min / ~70% to pass / SQL-first, else Python. Study Sections 3, 2, 4 hardest (they're the bulk); 7 is high-value and conceptual; 1, 5, 6 are vocabulary points. Use the trigger-word tips, eliminate, then apply the contrast rules. Never leave a blank.

DOMAIN 1

Databricks Intelligence Platform

6% of the exam · roughly 3 questions

Before you can ingest, transform, or govern anything, you have to understand the ground you are standing on. This chapter builds that foundation: what a lakehouse actually is, how Databricks splits work between its own cloud and yours, how Delta Lake turns ordinary files into reliable tables, how Unity Catalog governs everything, and how the different kinds of compute are priced. It is only 6% of the exam, but nearly every later question quietly assumes you already know this material.

Subdomain 1.1 — Understand the core components of the Databricks Data Intelligence Platform

> *Understand the core components of the Databricks Data Intelligence Platform, such as its architecture, Delta Lake, and Unity Catalog.*

1 · Lakehouse Architecture

Databricks is a **lakehouse**: the cheap, open storage of a data lake combined with the reliability and performance features of a data warehouse (ACID, governance, fast SQL). Everything runs on top of your cloud (AWS/Azure/GCP) — Databricks does not store your data in a proprietary system.

> **Easy picture:** a data lake is a giant garage — throw anything in (cheap, flexible) but nothing's organized and two people rearranging at once cause chaos. A warehouse is a bank vault — neat, fast, secure, but expensive and structured-only. The lakehouse is a well-organized garage with a labeling system: lake-cheap storage, vault-grade reliability on top. That labeling system is **Delta Lake + Unity Catalog**.

	Data lake	Data warehouse	Lakehouse
Data types	Anything	Structured only	Anything
Cost	Cheap object storage	Expensive proprietary	Cheap object storage
ACID reliability	No	Yes	Yes (Delta Lake)
BI/SQL performance	Poor	Great	Great (Photon)
ML on raw data	Yes	Hard	Yes

> **Exam tip:** "why lakehouse?" → one platform for BI and ML, warehouse reliability at data-lake cost, no data copies between systems. (Official sample answer: Delta Lake ACID + time travel, governed by Unity Catalog = single source of truth for AI and BI.)

The two planes

Plane	Who runs it	What lives there
Control plane	Databricks' account	Workspace UI, notebooks, job scheduler, cluster manager, Unity Catalog metadata, REST APIs

Plane	Who runs it	What lives there
Compute plane (classic)	Your cloud account	Clusters/warehouses that run your code; read/write your storage
Compute plane (serverless)	Databricks' account	Instant-start managed compute & SQL warehouses

Trap — where does customer data live? Your data always stays in your cloud object storage (S3/ADLS/GCS). The control plane only holds metadata and configuration. "Where does X happen?": permissions = control plane/UC, execution = compute plane, data = your cloud storage.

Identities

Three principal types, synced from account to workspaces: **users** (humans, by email), **groups** (collections — the recommended grant target), **service principals** (robot identities for automation). Production jobs should run as a service principal, not a person, so pipelines don't break when someone leaves.

2 · Delta Lake

Delta Lake is the default table format: an open-source layer that turns a folder of Parquet files into a reliable table by adding a transaction log (`_delta_log`: JSON commits + Parquet checkpoints). Delta is a **format**, not a service.

Feature	What it gives you
ACID transactions	Writes fully succeed or fully fail; readers never see half-written data
Time travel	`SELECT * FROM t VERSION AS OF 3` / `TIMESTAMP AS OF ...`
Schema enforcement	Wrong-schema writes are rejected
Schema evolution	`mergeSchema` adds new columns when allowed
Audit history	`DESCRIBE HISTORY t` — who/what/when

```
-- core commands you must know cold
CREATE TABLE sales (id INT, amount DOUBLE, region STRING);   -- Delta by default
UPDATE sales SET amount = 100 WHERE id = 1;                   -- new version
DELETE FROM sales WHERE id = 2;
DESCRIBE HISTORY sales;                                       -- list versions
SELECT * FROM sales VERSION AS OF 1;                          -- time travel
RESTORE TABLE sales TO VERSION AS OF 1;                       -- roll back (new commit)
MERGE INTO sales t USING updates s ON t.id=s.id               -- upsert pattern
  WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *;
OPTIMIZE sales;                                               -- compact small files
VACUUM sales;                                                 -- delete unreferenced files >7 days
```

Trap — updates don't edit files in place: an UPDATE writes new files and marks old ones removed in the log. That's why time travel works — and why VACUUM is needed to reclaim storage. After VACUUM, time travel beyond the retention window (default 7 days) is impossible; the files are physically gone.

> **Exam tip:** recover accidentally-deleted rows → `RESTORE` / time travel (fast), never re-ingest. RESTORE creates a new version equal to the old content; history is preserved. On new tables prefer Liquid Clustering (`CLUSTER BY`) over partitioning + ZORDER.

Delta features to recognize (name-level)

Feature	One-line meaning
Change Data Feed (CDF)	Records row-level inserts/updates/deletes so downstream jobs can read just what changed (`table_changes()`)
Deletion vectors	Mark rows deleted without rewriting the whole file — much faster DELETE/UPDATE/MERGE
UniForm	Writes Iceberg metadata alongside Delta so other engines (e.g. Snowflake) can read the table — no copy
DEEP vs SHALLOW CLONE	DEEP = full independent copy (backups/testing); SHALLOW = pointer to the same files, instant & cheap
OPTIMIZE + ZORDER	Compact small files; ZORDER co-locates data by a column for data skipping (superseded by Liquid Clustering on new tables)

> **Exam tip:** at Associate level you mostly need to recognize these names and what problem each solves — you won't be asked to configure them in depth.

Recovery & conversion operations (Delta / UC)

Command	What it does
`UNDROP TABLE cat.sch.t`	Recovers an accidentally-dropped managed table within 7 days (before VACUUM permanently deletes files)
`ALTER TABLE t SET MANAGED`	Converts external → managed in place (keeps table identity), but copies data to UC storage — the original external files remain until you delete them manually
DEEP CLONE vs SET MANAGED	DEEP CLONE makes a new table (copy, for sandboxes/cross-catalog); SET MANAGED is the in-place conversion of the same table
`CREATE TABLE … AS SELECT` (CTAS)	Create + populate a table from a query in one step

3 · *Unity Catalog (UC)*

UC is the platform's central governance layer: one place for access, audit, lineage, and discovery across all workspaces in a region. Everything is addressed with a three-level namespace: `catalog.schema.table`.

```
Metastore (one per region)
 └─ Catalog                # often dev / staging / prod
      └─ Schema            # business domains: finance, marketing…
           └─ Tables · Views · Volumes · Functions · Models
```

Object	What it is
Table (managed/external)	Delta tabular data (managed = UC owns storage; external = your path)

Object	What it is
View	Saved query, computed at read time
Volume	Governed storage for non-tabular files (`/Volumes/cat/sch/vol/…`)
Function	UDF (also used for row filters / column masks)
Model	ML model registered & governed in UC

UC gives centralized ACLs (grant once, applies everywhere), automatic lineage, auditing via system tables, and discovery. It replaced the old per-workspace Hive metastore (still visible as the `hive_metastore` catalog, ungoverned).

> **The contrast people get wrong — Volumes vs DBFS:** DBFS is legacy and ungoverned (anyone in the workspace can read it). Volumes are the modern replacement — governed by UC, so you `GRANT` / `REVOKE` access and every read is audited. "Store raw files under governance" → always a Volume, never DBFS.

> **Missed-question focus — volume privileges are READ VOLUME / WRITE VOLUME:** volumes hold files, so their privileges are file privileges: GRANT READ VOLUME to read files, GRANT WRITE VOLUME to upload/write files. SELECT and INSERT are for table rows, not volumes. So "let a service principal upload files to a volume" → WRITE VOLUME, never `INSERT`.

> **Exam tip:** "one consistent set of permissions across several workspaces" → Unity Catalog (one metastore, centralized governance) — not cluster ACLs, not per-workspace Hive metastores. System tables: `system.billing.usage` (cost/DBUs), `system.access.audit` (who did what).

Subdomain 1.2 — Understand Databricks Data Intelligence Platform's compute services

> *Understand Databricks Data Intelligence Platform's compute services, including their characteristics, limitations, and cost models, and select the most suitable option for each workload use case.*

4 · Compute Services & Cost Models

Compute type	Built for	Notes / cost
All-purpose cluster	Interactive dev, notebooks, ad-hoc	Highest DBU rate; stays up; multi-user. Wasteful for scheduled work
Jobs compute (job cluster)	Scheduled production pipelines	Cheaper DBU rate; created per run, terminates after. Not for interactive use
SQL warehouse	SQL analytics / BI dashboards	SQL only; Serverless/Pro/Classic; T-shirt sized; always Photon
Serverless compute	Fast zero-config execution	Starts in seconds, auto-scales, managed; less low-level control

The cost model — DBUs

Databricks bills in DBUs (a normalized unit of processing per hour). Total = DBUs × rate (depends on compute type + edition) + cloud VM cost (classic). For serverless, the VM cost is bundled into a higher DBU rate.

```
Job: 2 h × 10 DBU/h × $0.15 (jobs rate)   = $3.00   (+ cloud VM, classic)
Same as all-purpose: 2 × 10 × $0.40       = $8.00   (~2.7× more for the same work)
```

Cost levers (frequent exam answers): use jobs compute for scheduled work (cheaper); enable auto-termination on interactive clusters; use autoscaling; use serverless SQL warehouses for BI; Photon (C++ vectorized engine) costs more per hour but usually finishes faster so total cost drops (SQL/DataFrame ops only — not Python UDFs).

Guardrails & startup

Feature	Purpose
Cluster policy	Admin template limiting VM types, max workers, forced auto-termination/tags — prevents bad configs at creation
Cluster pool	Pre-warmed idle VMs → clusters start in seconds (idle pool VMs cost no DBUs, but cloud still bills the VMs)
Spot instances	Cheap interruptible VMs (driver on-demand, workers spot) for fault-tolerant batch
Access modes	Standard (shared, isolated, full UC) vs Dedicated (single user/group)

The contrast people get wrong — enforce vs react: a cluster policy prevents oversized clusters at creation (proactive). A script that kills expensive clusters is reactive — money's already spent. "Admin wants to enforce/restrict/standardize compute" → cluster policy. Auto-termination is a per-cluster setting; there's no global toggle, so admins fix/cap it in a policy.

Exam tip: "instant startup, no infrastructure to manage" → serverless. "VMs in the customer's cloud subscription" → classic. "Many BI analysts, fast startup, many concurrent users, avoid over-scaling" → a high-concurrency / SQL warehouse with autoscaling (official sample answer C). "Slow classic cluster starts for frequent short jobs" → cluster pools.

Databricks Runtime (DBR) — recognize

A cluster runs a Databricks Runtime: a versioned image bundling Spark, Delta Lake, and common libraries. LTS (long-term support) versions are the safe choice for production; ML runtimes add machine-learning libraries; serverless manages the runtime for you. Exam-level: know that DBR = the versioned engine image, and LTS = the stable, production-safe track.

Worked Examples & Field Notes

The tables above give you the vocabulary. Now let's use it the way the exam does — as little stories where you have to pick the right piece of the platform. Read each scenario, cover the answer, and decide before you peek.

Walkthrough: "Where does everything actually happen?"

This is the single most reused idea in Chapter 1, so let's trace a query end to end. Imagine you press Run on a notebook cell that reads a table:

```
# you run this in a notebook
spark.read.table("prod.finance.transactions").filter("amount > 1000").count()
```

Here is the journey, and who owns each step. The request first hits the **control plane** (Databricks' side): the scheduler and Unity Catalog check "is this person allowed to read this table?" If yes, the actual work is handed to your **cluster in the compute plane**, which reaches directly into your **cloud object storage**, reads the Parquet files behind the Delta table, filters, counts, and returns the number. Three actors, three jobs: **control plane = permission & coordination, compute plane = execution, cloud storage = the data itself.**

> **Exam tip:** any question shaped like "where does X live / happen?" is answered by that trio. Permissions and metadata → control plane. Running code → compute plane. The bytes of your data → your cloud storage, never Databricks' servers.

Coffee break. *A lakehouse walks into a bar and orders one drink for both its BI friend and its ML friend. The bartender says "one copy of the data, coming right up." The data warehouse in the corner, who ordered two of everything and paid triple, did not find this funny.*

Walkthrough: choosing compute without lighting money on fire

Compute questions almost always hide a cost trap. Same workload, three framings:

Scenario	Right compute	Why the others lose
Nightly ETL that must "just run" cheaply	Jobs compute (job cluster)	All-purpose is ~2–3× the DBU rate and stays up idle
A data scientist poking at data all afternoon	All-purpose cluster (with auto-termination)	Jobs compute tears down after each run — no interactive session
50 analysts firing ad-hoc SQL at 9am	Serverless SQL warehouse w/ autoscaling	A single big cluster queues; all-purpose can't serve SQL concurrency well
"Instant start, nothing to configure"	Serverless	Classic clusters boot VMs for minutes

```
# the cost formula, made concrete
Job on jobs compute:   2 h × 10 DBU/h × $0.15   = $3.00   (+ your cloud VM bill)
Same on all-purpose:   2 h × 10 DBU/h × $0.40   = $8.00   (~2.7× for identical work)
```

> **Easy picture:** all-purpose compute is a rental car you keep parked outside your house all month "just in case." Jobs compute is calling a taxi only when you actually need to go somewhere. Both get you there; only one shows up on your statement as a small fortune.

Real talk. *Ninety percent of surprise Databricks bills are one all-purpose cluster that someone spun up for a "quick test" in March and never turned off. Auto-termination exists because we are all that someone.*

Walkthrough: Delta Lake time travel saves your weekend

Suppose a bad job just overwrote your sales table with garbage. Panic is optional, because Delta never truly overwrites — it writes new files and keeps a log.

```
DESCRIBE HISTORY sales;                    -- find the last good version, say v41
SELECT * FROM sales VERSION AS OF 41;      -- peek to confirm it's clean
RESTORE TABLE sales TO VERSION AS OF 41;   -- roll back — creates v43, doesn't erase history
```

Notice you did not re-ingest from the source, dig through backups, or file an incident. You asked the table what it looked like an hour ago and told it to go back. That is the whole pitch of the lakehouse in one command.

> **Trap:** this magic has an expiry date. VACUUM physically deletes files older than the retention window (default 7 days). Run VACUUM, and time travel beyond that window is gone forever — the files simply do not exist anymore. Time travel is a safety net, not an infinite undo.

One-Minute Recap

Lakehouse = lake-cheap storage + warehouse reliability (Delta) + governance (UC); one platform for BI & ML, no data copies. Two planes: control plane (Databricks: UI, scheduler, metadata) vs compute plane (classic = your VMs, serverless = Databricks' VMs); data always in your cloud storage. Delta Lake = Parquet + `_delta_log` → ACID, time travel (`VERSION AS OF` / `RESTORE`), schema enforcement; UPDATE writes new files (VACUUM reclaims, 7-day default kills older time travel); MERGE = upsert; OPTIMIZE compacts. Unity Catalog = metastore → catalog → schema → object (`catalog.schema.table`); centralized grants/lineage/audit across workspaces; Volumes replace DBFS for governed files. Compute: all-purpose (interactive, pricey) · jobs compute (scheduled, cheap) · SQL warehouse (BI, Photon) · serverless (instant, zero-config). Cost = DBUs × rate + VMs (classic); save via jobs compute, auto-termination, autoscaling, serverless, Photon. Cluster policy = enforce configs (proactive); pools = fast starts. Principals: users/groups/service principals; prod runs as a service principal.

DOMAIN 1 · TARGETED DEEP-DIVE

Compute & Schema: Closing Your Domain-1 Gaps

Insert for §4 (Compute) and §2 (Delta) · built from the exact patterns you missed.

Your practice results clustered in one place: picking compute by familiarity instead of by workload. Every miss below is the same reflex — reaching for the tool you know rather than the one the scenario describes. This deep-dive rebuilds the decision rules so the right answer becomes automatic. Read it right after §4 of this chapter; it uses the same vocabulary.

> **The one habit to fix.** The exam never rewards "which compute am I comfortable with?" It rewards "which compute is built for this workload's shape?" Shape = who uses it (one person vs many), what runs on it (SQL / BI / pandas / R / Spark), how big the data is, and who pays. Match the shape, not your habits.

1 · *Compute Type Selection — match the workload, not your comfort zone*

Chapter 1 §4 gave you the four compute families. The exam tests them as disambiguation: two options both "work," but only one is purpose-built. Here is the full decision table, with the traps you hit.

When the scenario is about…	Correct compute	Why the tempting wrong answer loses
A third-party BI tool / interactive dashboards, many concurrent queries	SQL Warehouse (Serverless/Pro)	A High-Concurrency / all-purpose cluster is general-purpose; the SQL Warehouse is purpose-built for BI — native connectors, concurrency scaling, always-Photon.

When the scenario is about…	Correct compute	Why the tempting wrong answer loses
Small dataset (MBs), pandas / scikit-learn, non-distributed libraries	Single Node cluster	A SQL Warehouse runs SQL only — it can't run pandas/sklearn. Distributed clusters waste money on a 50 MB job that never leaves one machine.
Interactive notebook development, ad-hoc exploration	All-purpose cluster (auto-terminate)	Jobs compute tears down after each run — no interactive session.
Scheduled production ETL that "just runs"	Jobs compute (job cluster)	All-purpose is ~2–3× the DBU rate and idles between runs.
"Instant start, zero infra to manage"	Serverless	Classic clusters boot VMs for minutes.

Exam tip — the two you missed, side by side. "BI tool connecting for dashboards" → SQL Warehouse, never a High-Concurrency cluster (that's the old general-purpose answer). "pandas + scikit-learn on 50 MB" → Single Node, never a SQL Warehouse (SQL Warehouses can't run Python libraries at all). Both hinge on what language/engine the workload actually needs.

Easy picture. A SQL Warehouse is a dedicated espresso machine — it does one drink perfectly for a crowd. A Single Node cluster is a single kitchen counter — perfect when one cook is making one small dish with their own tools (pandas/sklearn). You wouldn't order espresso to chop vegetables, and you wouldn't buy a commercial espresso bar to make one cup.

Trap — "distributed is always better." For a 50 MB pandas/sklearn job, distribution is pure overhead: the data fits in one machine's memory and the libraries aren't Spark-aware. Single Node (one driver, no workers) is the cheapest correct answer. Bigger/parallel is a distractor here.

2 · *Photon vs Serverless — engine, not deployment*

This one caught you because both sound like "make it faster." They live on different axes entirely.

	Photon	Serverless
What it is	A query execution engine (C++ vectorized) that runs your SQL/DataFrame ops faster	A compute deployment model — Databricks manages the VMs, instant start
Fixes…	Slow SQL/DataFrame execution: joins, aggregations, scans	Slow startup / infra management overhead
Axis	How fast the query runs	Where/how the compute is provisioned

Exam tip. "Heavy SQL ETL with complex joins is slow — how do you accelerate execution?" → Photon. Serverless changes who runs the VMs, not how fast a join executes. They're not alternatives; you can even have both. Pick Photon whenever the pain is query speed on SQL/DataFrame work (not Python UDFs — Photon doesn't accelerate those).

3 · *Cluster Access Modes — capability, not preference*

Two of your misses (multi-user collaboration, and R) are the same table read two ways. Access mode is chosen by capabilities the workload requires, not by what feels safe. Learn this matrix cold.

Access mode	Who it's for	Unity Catalog	Languages	R / init scripts / ML
Standard (formerly Shared)	Multiple users on one cluster, with per-user isolation	✓ Full UC, enforced per user	SQL, Python, Scala	✗ No R
Dedicated (formerly Single User)	One user or one group; personal dev, ML	✓ Full UC, as the single identity	SQL, Python, Scala, R	✓ R, init scripts, ML runtime
No Isolation Shared (legacy)	Legacy multi-user; being retired	✗ No UC governance	SQL, Python, Scala, R	—

The contrast people get wrong — Standard vs Dedicated. Decide on two questions: (1) How many people share this cluster? Many with UC governance → Standard/Shared. (2) Does the workload need R, init scripts, or ML runtime? → Dedicated/Single User (Standard doesn't support R). That's the whole decision.

Exam tip — your two misses resolved. "Five engineers collaborate on one cluster with UC governance" → Standard (Shared), because it enforces UC per user for many users on the same cluster. Single User/Dedicated binds the whole cluster to one identity — wrong for a shared team. "Configure a cluster for a data scientist using R with UC" → Dedicated, because Standard can't run R and No Isolation Shared has no UC. Don't pick No Isolation Shared for anything governed — it's the ungoverned legacy trap.

Trap — "Shared = safe default for everything." Shared/Standard is right for multi-user work, but it cannot run R or init scripts. The moment a scenario says R, init scripts, or ML runtime, the answer flips to Dedicated even if only "one user" is mentioned.

4 · *Cluster Policy — admin-enforced, not a user workaround*

Your Q1 miss chose an access mode to control runaway cost. Access mode is a per-user setting; cost governance is an admin control. Different layer, different tool.

The contrast people get wrong — policy vs user setting. "Engineers keep leaving expensive clusters running overnight" is a governance problem, so the answer is a cluster policy that forces auto-termination, caps VM size/workers, and standardizes tags — enforced at cluster creation for everyone. Access mode, auto-termination toggled by hand, or a cleanup script are all per-user or reactive; they don't enforce anything org-wide.

Exam tip. Any question with an admin wanting to "enforce / restrict / standardize / prevent" compute configs → cluster policy (proactive, at creation). If the fix is a setting an individual user picks (access mode, manual auto-terminate), it's the wrong layer for a fleet-wide cost problem.

5 · *Compute Reliability Features — the two "which feature?" traps*

Instance-type unavailability → Fleet instance types. When jobs fail because a specific cloud instance type is out of capacity, the fix is a feature that auto-selects from a pool of equivalent instance types — not switching compute family.

Exam tip. "Jobs fail because the requested instance type is unavailable" → Fleet instance types (Databricks picks any equivalent available type). Switching to All-Purpose Compute doesn't solve availability — it's a different compute type, same capacity risk.

Disk exhaustion during big shuffles → Autoscaling local storage. Massive shuffles spill to local disk; when the disk fills, the job dies. The fix is a storage feature, not an availability feature.

Exam tip. "Complex transformation with massive shuffles runs out of disk" → autoscaling local storage (Databricks attaches more disk as spill grows). Falling back to On-Demand instances is about VM availability/price, not shuffle disk space — wrong axis.

Trap — availability vs storage. Two different problems wearing similar clothes. Instance type not available → Fleet. Disk filling mid-shuffle → autoscaling local storage. Spot/On-Demand fallback answers neither; it's a price/availability lever.

6 · *Delta Schema Evolution — write-time merge, not manual DDL*

For an automated ingestion job appending JSON whose schema drifts, you want the schema to evolve at write time, not a human running DDL after every change.

The contrast people get wrong — mergeSchema vs ALTER TABLE. `ALTER TABLE … ADD COLUMNS` is manual DDL — someone has to notice the new column and run it, which breaks an automated pipeline. `mergeSchema` evolves the schema automatically on the write itself. For hands-off ingestion, always the write-time option.

```
# DataFrame write: evolve schema automatically as new columns appear
(df.write.format("delta")
   .option("mergeSchema", "true")      # add new columns at write time
   .mode("append")
   .saveAsTable("main.bronze.events"))

# Or set it once for the session so every append auto-evolves:
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

Trap. `mergeSchema` adds new columns; it does not silently drop or retype existing ones (that's still blocked by schema enforcement). "Automated pipeline, new fields appear over time, no manual steps" → `mergeSchema`, not `ALTER TABLE`.

One-Minute Recap — Domain 1 gap closers

Compute by workload shape: BI tool/dashboards → SQL Warehouse (not High-Concurrency cluster); pandas/sk-learn on small data → Single Node (not SQL Warehouse); interactive dev → all-purpose; scheduled ETL → jobs compute; instant/no-infra → serverless. Photon = execution engine (speeds SQL/DataFrame); Serverless = deployment model (instant VMs) — different axes. Access modes: many users + UC → Standard/Shared (no R); R / init scripts / ML → Dedicated/Single User; never No Isolation Shared for governed work. Cost governance = admin cluster policy (enforces auto-termination at creation), not a user access-mode setting. Reliability: instance type unavailable → Fleet; disk full mid-shuffle → autoscaling local storage. Schema drift in an automated append → mergeSchema (write-time), not manual ALTER TABLE.

DOMAIN 2

Data Ingestion and Loading

21% of the exam · roughly 9 questions

Ingestion is the largest slice of the exam after transformation, and the questions almost always come down to a single decision: given this situation, which tool should move the data? This chapter walks through reading the common file formats, the crucial split between batch and streaming reads, Auto Loader for incremental file ingestion, COPY INTO for SQL batch loads, and the managed connectors of Lakeflow Connect — with the rules for choosing between them.

Subdomain 2.1 — Enable and detail data ingestion patterns

> *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

The Shape of This Domain

Domain 2 is the biggest slice of the exam. It's about getting data into the lakehouse and manipulating it with the two core APIs. Almost every question reduces to one of four things: which reader (batch vs stream), which format + options, DataFrame vs SQL syntax, or Auto Loader details.

Theme	What you must know cold
Reading sources	`spark.read.format(...).option(...).load(...)` for CSV/JSON/Parquet/Delta
Batch vs streaming	`spark.read` (finite, one-shot) vs `spark.readStream` (continuous/incremental)
Auto Loader	`cloudFiles` format: incremental file ingestion + schema inference & evolution
DataFrame API	create, inspect, transform DataFrames
Spark SQL	temp views, `spark.sql(...)`, catalog operations
Dev patterns	notebooks, magics, Databricks Connect

> **Two traps the exam loves:** (1) confusing `spark.read` with `spark.readStream` — batch vs stream; (2) missing Auto Loader schema-inference details (where the schema is stored, how evolution behaves). Both are covered in depth below.

Reading Data Sources — the universal pattern

Every batch read follows the same skeleton. Learn the skeleton, then just swap the format string and options.

```
df = (spark.read                        # 1) the batch DataFrameReader (finite data)
       .format("csv")                   # 2) source format: csv | json | parquet | delta
       .option("header", "true")        # 3) format-specific options (0..n of them)
       .schema(my_schema)               # 4) optional explicit schema (recommended in prod)
       .load("/path/to/data"))          # 5) the path (or .table("cat.sch.tbl") for a table)
```

Line by line: `spark.read` returns a `DataFrameReader` configured for batch. `.format()` names the source. `.option()` sets one key/value tweak (chain many, or use `.options(dict)`). `.schema()` supplies a known schema so Spark skips inference. `.load(path)` triggers the read and returns a DataFrame.

CSV — *the most option-heavy format*

```
df = (spark.read.format("csv")
      .option("header", "true")          # first row = column names (else _c0,_c1,…)
      .option("inferSchema", "true")     # scan data to guess types (extra pass = slower)
      .option("sep", ",")                # delimiter (use "\t" for TSV)
      .option("nullValue", "NA")         # treat "NA" as null
      .option("mode", "PERMISSIVE")      # bad rows: PERMISSIVE|DROPMALFORMED|FAILFAST
      .load("/Volumes/main/raw/customers/"))

# shorthand: format+load collapse into a helper
df = spark.read.csv("/Volumes/main/raw/customers/", header=True, inferSchema=True)
```

Easy picture: CSV is a plain text spreadsheet with no memory of types — every value is just text until you tell Spark otherwise. `header` hands it the column names; `inferSchema` makes Spark taste every column to guess int/double/string. That "tasting" is a whole extra scan of the file.

Trap — inferSchema costs a pass: `inferSchema=true` reads the data twice (once to infer, once to load), which is slow on big files. In production, supply an explicit `.schema(...)` instead — faster and stable (types don't drift when the data changes).

JSON

```
df = (spark.read.format("json")
      .option("multiLine", "true")     # true = one JSON object spanning many lines / an array
      .load("/Volumes/main/raw/events/"))
```

Trap — JSON default is one-object-per-line (JSONL): Spark expects newline-delimited JSON by default (`{...}\n{...}`). A pretty-printed file or a top-level array needs `multiLine=true` , or every row comes back corrupt/null. Nested JSON becomes struct / array columns you later flatten with dot-notation and `explode` .

Parquet — schema built in

```
df = spark.read.format("parquet").load("/Volumes/main/raw/orders/")
df = spark.read.parquet("/Volumes/main/raw/orders/")     # shorthand
```

Parquet is columnar + self-describing: it stores the schema and column stats in its own footer, so no inferSchema needed and reads are fast (Spark skips columns and row-groups it doesn't need).

Delta — Parquet with a transaction log

```
df = spark.read.format("delta").load("/Volumes/main/silver/orders")       # by path
df = spark.read.table("main.silver.orders")                               # by UC name (preferred)
```

The contrast people get wrong — Parquet vs Delta: Delta is Parquet data files plus a `_delta_log` transaction log. That log adds ACID, time travel, schema enforcement and fast metadata. Rule: reading a governed lakehouse table → delta (or just `.table()`); reading a raw dump of column files with no log → parquet. Delta is the default format when you `saveAsTable` .

Format	Needs inferSchema?	Self-describing?	Typical use
CSV	Yes (or explicit schema)	No	Raw text dumps, exports
JSON	Auto-parses; watch multiLine	Partly (no fixed types)	Semi-structured/API data
Parquet	No	Yes (footer schema)	Efficient columnar storage
Delta	No	Yes (+ transaction log)	Lakehouse tables (default)

Exam tip: `.load(path)` reads files at a path; `.table("cat.sch.tbl")` reads a registered Unity Catalog table. Both return an identical DataFrame. Writing mirrors reading: `df.write.format("delta").mode("overwrite").saveAsTable(...)`.

Batch vs Streaming — spark.read vs spark.readStream

This is the most-tested distinction in the domain. Same builder pattern, different entry point, completely different behavior.

	spark.read (batch)	spark.readStream (streaming)
Data view	A finite snapshot — reads what exists now, then stops	An unbounded stream — keeps processing new data as it arrives
Returns	A DataFrame	A streaming DataFrame
Write with	`df.write.save/saveAsTable`	`df.writeStream ... .start()`
Tracks progress?	No — re-reads everything each run	Yes — a checkpoint remembers what's been processed
Use for	One-off / scheduled full or partitioned loads	Continuous or incremental ingestion (only new data)

```
# BATCH: read everything at the path, once
batch_df = spark.read.format("json").load("/Volumes/main/raw/events/")
batch_df.write.mode("overwrite").saveAsTable("main.bronze.events")

# STREAM: process only new files as they land, forever (or until stopped)
stream_df = spark.readStream.format("json").load("/Volumes/main/raw/events/")
(stream_df.writeStream
     .option("checkpointLocation", "/Volumes/main/_chk/events")          # progress bookmark
     .toTable("main.bronze.events"))
```

Easy picture: `spark.read` is taking a photo of a folder — you get exactly what's there at that instant. `spark.readStream` is a security camera — it keeps rolling and reacts to each new file that appears, remembering where it left off via the checkpoint.

Trap — the checkpoint is mandatory for streams: a streaming write needs a `checkpointLocation`. It stores which files/offsets are already processed, so a restart resumes instead of re-ingesting everything. No checkpoint → no exactly-once guarantee. Batch reads have no checkpoint (they just re-read).

> **Exam tip:** "process files as they arrive / only new data / incremental" → readStream (usually Auto Loader). "Load the current contents once / a scheduled full refresh" → read. If you see `.writeStream` or `checkpointLocation`, it's a stream.

Streaming triggers & output modes (recognize these)

Trigger	Behavior
`trigger(availableNow=True)`	Process all available data now in micro-batches, then stop — ideal for scheduled incremental jobs
`trigger(processingTime="30 seconds")`	Run a micro-batch every fixed interval (continuous-ish, always on)
default (no trigger)	Micro-batches as fast as possible, runs continuously until stopped

Output mode (writeStream)	Writes
append (default)	Only new rows — for straightforward ingestion
complete	The full updated result table every batch — for aggregations
update	Only rows that changed this batch

> **Exam tip:** availableNow = "catch up then stop" (batch-like, cheap); a processingTime or default trigger keeps the stream running. Aggregations that must re-emit totals use complete mode; plain ingestion uses append.

> **Late & out-of-order data (recognize):** in streaming, events can arrive out of order or late. A watermark (`.withWatermark("event_time", "10 minutes")`) tells Spark how long to wait for late events before finalizing a windowed aggregation and dropping stragglers — it bounds state so the stream doesn't grow forever. Exam-level: know that watermarks handle late data in windowed/stateful streaming.

Core APIs & Dev Patterns (foundations)

Ingestion lands data; the two core APIs and the notebook toolkit are how you then read and shape it. These foundations underpin every subdomain below.

DataFrame API Fundamentals — creating & inspecting

```
from pyspark.sql import Row

# create from literals (handy for tests)
df = spark.createDataFrame([(1,"EU"),(2,"US")], ["id","region"])

# inspect
df.printSchema()        # column names + types (the tree view)
df.show(5, truncate=False)    # first 5 rows, full width
df.columns              # list of column names
df.dtypes               # [(name, type), …]
df.count()              # number of rows (an ACTION - triggers compute)
```

The contrast people get wrong — transformations vs actions: transformations (`select` , `filter` , `withColumn` , `join` , `groupBy`) are lazy — they build a plan but run nothing. Actions (`show` , `count` , `collect` , `write`) trigger execution. Nothing computes until an action fires. This laziness lets Spark optimize the whole chain (e.g. push filters down).

Core transformations (batch or stream, same API)

```
from pyspark.sql.functions import col, when

result = (df
   .select("id", "region")                          # pick columns
   .filter(col("region") == "EU")                   # keep rows (alias: where)
   .withColumn("zone", when(col("region")=="EU","euro").otherwise("other"))
   .withColumnRenamed("id", "customer_id")
   .drop("region"))
```

Exam tip: the DataFrame API is identical whether the source is batch or streaming — you write the same select/filter/withColumn. Only the read (`read` vs `readStream`) and write (`write` vs `writeStream`) endpoints differ. That's why "learn the transformations once" pays off.

Spark SQL Foundations

Anything you can do in the DataFrame API you can do in SQL, and vice-versa. The bridge is a temp view.

```
# register a DataFrame as a SQL-queryable view (session-scoped, not stored)
df.createOrReplaceTempView("my_table")

# now query it with SQL; returns a DataFrame
result = spark.sql("""
     SELECT region, count(*) AS n
     FROM my_table
     WHERE status = 'active'
     GROUP BY region
""")
result.show()
```

View type	Scope / lifetime	Create with
Temp view	Current Spark session only	`createOrReplaceTempView`
Global temp view	All sessions on the cluster (`global_temp.name`)	`createOrReplaceGlobalTempView`
(Persistent) view	Stored in Unity Catalog, survives restarts	`CREATE VIEW cat.sch.name AS …`

The contrast people get wrong — temp view vs a real (UC) view: a temp view is an in-memory name for a DataFrame, gone when the session ends, invisible to other users. A persistent view lives in Unity Catalog, is governed, and everyone can query it. Rule: quick in-notebook SQL over a DataFrame → temp view; a reusable governed object for BI → `CREATE VIEW` .

```
# catalog / namespace operations you should recognize
spark.sql("USE CATALOG main")
spark.sql("USE SCHEMA silver")
spark.sql("SHOW TABLES")
```

```
spark.catalog.listTables()            # programmatic equivalent
spark.table("main.silver.orders")     # DataFrame from a table (same as read.table)
```

Exam tip: in a notebook you can flip languages per cell with magics: `%sql` runs a SQL cell, `%python` a Python cell, over the same temp views and tables. `spark.sql()` is the programmatic form of a `%sql` cell.

Notebook Development Patterns

Tool	What it does
`%sql`, `%python`, `%scala`, `%r`	Switch a cell's language; all share the same session/tables
`%md`	Markdown documentation cell
`%run ./other_nb`	Inline-execute another notebook (share functions/vars)
`dbutils.fs`	File system ops (`ls`, `cp`, `rm`) on volumes/DBFS
`dbutils.widgets`	Parameterize a notebook (values passed by a job)
`dbutils.notebook.run`	Run another notebook as a function, with parameters + a return value
`display(df)`	Rich, sortable, chartable table output

```
# parameterize a notebook so a job can pass values in
dbutils.widgets.text("run_date", "2026-07-01")
run_date = dbutils.widgets.get("run_date")
df = spark.read.table("main.bronze.orders").filter(col("order_date") == run_date)
```

The contrast people get wrong — %run vs dbutils.notebook.run: `%run` inlines another notebook into yours (its variables/functions become available — like an import). `dbutils.notebook.run` executes a notebook as a separate job with its own scope, takes parameters, and returns a value. Rule: share helper code → `%run`; orchestrate/parameterize a child task → `dbutils.notebook.run`.

Databricks Connect

Databricks Connect lets you run Spark code from a local IDE (VS Code, PyCharm) or app while the actual computation executes on a remote Databricks cluster/serverless. You build DataFrames locally; the work runs remotely.

Easy picture: it's a remote control for a cluster. Your laptop holds the buttons (your code and DataFrame plan); the heavy machinery (the Spark cluster) does the lifting in the cloud. Your laptop never has to be powerful.

Exam tip: recognize Databricks Connect as "author/run/debug Spark from your local IDE against remote Databricks compute". It's for development outside notebooks; it does not run Spark on your laptop — compute stays on the cluster. Modern versions are built on Spark Connect (a thin client/gRPC protocol).

Subdomain 2.2 — Use the COPY INTO command

> *Use the COPY INTO command to incrementally load files from cloud object storage (ADLS/S3/GCS) into Unity-Catalog–governed tables.*

COPY INTO is a SQL command that incrementally loads files from cloud object storage into a Unity Catalog table. It's idempotent: it remembers which files it already loaded and skips them on re-run — so you can safely schedule it.

```
COPY INTO main.bronze.orders
FROM '/Volumes/main/raw/landing/orders/'
FILEFORMAT = JSON                          -- CSV | JSON | PARQUET | AVRO …
FORMAT_OPTIONS ('inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');     -- allow new columns over time
```

The contrast people get wrong — COPY INTO vs Auto Loader: both ingest files idempotently into UC tables. COPY INTO is a simple SQL command, best for smaller, periodic batch loads (thousands of files, known/slowly-changing schema). Auto Loader (`cloudFiles`) scales to millions of files, is streaming, and shines with high volume, frequent arrival, or evolving schemas. Rule: "SQL, modest batch, retriggerable" → COPY INTO; "huge/continuous/streaming, schema evolution" → Auto Loader.

Exam tip: both COPY INTO and Auto Loader track already-processed files, so re-running does not duplicate data. That idempotency is the reason you pick them over a plain `spark.read` of the folder (which reloads everything).

COPY INTO — *details the exam tests*

Aspect	Detail
Idempotency	Tracks each loaded file in the Delta transaction log and skips it on re-run — safe to re-run, no duplicates
`COPY_OPTIONS('force'='true')`	Bypasses file tracking and reloads every matching file → duplicate rows. Also used to reset load history
Valid sources	Cloud object storage only: `s3://` , `abfss://` , `gs://` — not on-prem HDFS
Formats	CSV, JSON, PARQUET, AVRO, ORC, TEXT, BINARYFILE
Schema evolution	`COPY_OPTIONS('mergeSchema'='true')` adds new columns

Trap: `force = true` is the duplicate-maker — it re-ingests already-loaded files. Default (no force) is idempotent. If a question asks "re-running COPY INTO loads nothing new" → that's the normal idempotent behaviour.

Subdomain 2.3 — Use Auto Loader with schema enforcement and schema evolution

> *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity-Catalog–governed tables.*

Auto Loader is Databricks' recommended way to incrementally ingest files from cloud storage. It's a Structured Streaming source you select with `format("cloudFiles")` . It automatically detects new files, so you never re-scan the whole directory or hand-track what's been loaded.

```
df = (spark.readStream
        .format("cloudFiles")                                  # 1) THE Auto Loader source
        .option("cloudFiles.format", "json")                   # 2) underlying file format (json/
csv/parquet…)
        .option("cloudFiles.schemaLocation", "/Vol/_schema/events")      # 3) where inferred
```

```
schema is stored
         .option("cloudFiles.inferColumnTypes", "true")        # 4) infer real types, not all-
strings
         .load("/Volumes/main/raw/events/"))                   # 5) the source directory to watch

(df.writeStream
    .option("checkpointLocation", "/Vol/_chk/events")          # progress bookmark (separate from
schema loc)
    .trigger(availableNow=True)
# process all backlog now, then stop (batch-like)
    .toTable("main.bronze.events"))
```

Line by line: `format("cloudFiles")` picks Auto Loader; the real file type goes in `cloudFiles.format` (a classic gotcha — the outer format is always `cloudFiles`). `cloudFiles.schemaLocation` is a folder where Auto Loader persists the inferred schema so it can detect changes across runs. `inferColumnTypes` makes it infer int/double/etc. instead of reading everything as strings. `.load(dir)` is the directory to monitor.

Auto Loader detection modes

Auto Loader must discover new files. Two mechanisms:

Mode	How it finds files	Best for
Directory listing (default)	Lists the directory to find new files	Smaller/simple setups; no cloud config needed
File notification	Cloud events (SQS/Event Grid/PubSub) push new-file notifications	Very high file volumes / large directories — far more scalable

> **Exam tip:** huge directories where listing is slow → file notification mode (uses cloud notification services). Databricks can even switch from directory listing to notification automatically as volume grows. Enabling file events on the external location also speeds up file-arrival job triggers.

Schema inference & evolution — the detail the exam checks

Concept	What Auto Loader does
Schema inference	Samples files on first run, infers columns/types, and saves the schema to `schemaLocation`
Schema evolution	If a new column appears later, the stream fails once, records the new schema, and succeeds on restart (default mode `addNewColumns`)
Rescued data	Values that don't match the schema are kept in a `_rescued_data` column instead of being lost
schemaHints	You can pin specific column types to override inference

> **Easy picture:** Auto Loader keeps a guest list (the schema in `schemaLocation`). When a guest with a new name (new column) shows up, it stops at the door, adds the name to the list, and lets everyone in on the next try. Nothing unexpected is thrown away — odd values wait in the `_rescued_data` lobby.

> **The contrast people get wrong — schemaLocation vs checkpointLocation:** two different folders with two jobs. `schemaLocation` stores the inferred schema (so evolution can be detected). `checkpointLocation` stores streaming progress (which files are done). Both are needed for a robust Auto Loader stream; don't point them at the same purpose.

Schema enforcement vs schema evolution

> **The contrast people get wrong — enforcement vs evolution:** schema enforcement rejects data that doesn't match the table's schema (protects the table). Schema evolution updates the schema to accept new columns. Auto Loader does both: it enforces the known schema, and with the default `addNewColumns` evolution it stops once, records a new column, and succeeds on restart. Values that still don't fit land in the `_rescued_data` column instead of being lost.

schemaEvolutionMode	Behavior on a new column
`addNewColumns` (default)	Stream fails once, adds the column, succeeds on restart
`rescue`	New/unexpected data goes to `_rescued_data`; schema unchanged
`failOnNewColumns`	Stream fails and stays failed until you update the schema
`none`	Ignores new columns entirely

> **Exam tip:** memorize that Auto Loader = `cloudFiles`, the real format lives in `cloudFiles.format`, and it needs a `schemaLocation` (for inference/evolution) and a `checkpointLocation` (for progress). `trigger(availableNow=True)` runs it like a batch job: catch up on all new files, then stop — perfect for scheduled incremental jobs. Auto Loader is also the recommended file source inside Lakeflow Declarative Pipelines, where schema/checkpoint locations are managed for you.

Subdomain 2.4 — Configure Lakeflow Connect to reliably ingest data

> *Configure Lakeflow Connect to reliably ingest data from diverse enterprise sources into Unity-Catalog–governed tables.*

Lakeflow Connect is Databricks' newer managed ingestion layer: point-and-click connectors that pull data from enterprise apps and databases straight into Unity Catalog, with governance built in. It's generally available for connectors like Salesforce, Workday, and SQL Server, with a large and growing catalog of native connectors.

Standard vs managed connectors

Connector type	What it is	Examples
Standard connectors	Ingest from cloud storage / common file & message sources you configure	Cloud object storage, Kafka-style streams
Managed connectors	Fully-managed, point-and-click pipelines from SaaS apps & databases	Salesforce, Workday, SQL Server (all GA), + 100s more

> **The contrast people get wrong — Auto Loader vs Lakeflow Connect:** Auto Loader ingests files from cloud object storage (you write the stream). Lakeflow Connect ingests from SaaS apps and databases via managed connectors (little/no code, UI-driven), landing governed tables in Unity Catalog. Rule: files in a bucket → Auto Loader; Salesforce/Workday/SQL Server/app data → Lakeflow Connect.

> **The contrast people get wrong — Connection vs pipeline (where the destination is set):** a Lakeflow Connect setup has two parts. The Connection object holds the source credentials (how to reach Salesforce). The destination catalog and schema are set in the ingestion pipeline configuration — not in the Connection, and not via a pre-

start USE catalog/schema SQL script. Rule: source auth → Connection; where the data lands (destination catalog/schema) → pipeline details.

Exam tip: at Associate level, recognize the positioning: Auto Loader = code-based file ingestion; Lakeflow Connect = managed connector-based ingestion from external systems; both land data under Unity Catalog governance.

Subdomain 2.5 — Use JDBC/ODBC or REST clients in notebooks

> *Use JDBC/ODBC or REST clients in notebooks to land data into cloud storage or directly into Unity-Catalog–governed tables, usually orchestrated and scheduled with Lakeflow Jobs.*

When there's no connector, engineers pull data in code inside a notebook, usually orchestrated by a Lakeflow Job:

```
# JDBC: read from an external database directly into a DataFrame
df = (spark.read.format("jdbc")
       .option("url", "jdbc:postgresql://host:5432/db")
       .option("dbtable", "public.customers")
       .option("user", user).option("password", pw)
       .load())

# REST API: fetch with requests, then land as a table
import requests
data = requests.get("https://api.example.com/v1/data", timeout=30).json()
spark.createDataFrame(data).write.saveAsTable("main.bronze.api_data")
```

Exam tip: JDBC/ODBC = connect to external databases; REST clients = call web APIs. Both typically land data into cloud storage or UC tables and are scheduled/orchestrated with Lakeflow Jobs. Use these when no standard/managed connector exists.

JDBC — parallel reads, pushdown & the exam details

```
# pushdown: put a WHERE inside a subquery-as-dbtable so the DB filters BEFORE transfer
.option("dbtable", "(SELECT * FROM orders WHERE order_date > '2024-01-01') AS q")

# parallel read: numPartitions is IGNORED unless you also give the other three
.option("partitionColumn", "id").option("lowerBound", 1)
.option("upperBound", 1000000).option("numPartitions", 8)
```

Point	What to know
JDBC vs ODBC	JDBC = Spark's Java connector to read from external DBs. ODBC = protocol BI tools (Tableau/Power BI) use to connect to Databricks (Simba driver)
Parallel reads	`partitionColumn` + `lowerBound` + `upperBound` are required with `numPartitions`; miss any and Spark uses a single partition
Pushdown	Wrap a query in `dbtable` so the filter runs on the DB server, moving less data
fetchsize	Rows fetched per round trip (tune for read throughput)

Subdomain 2.6 — Prioritize between Auto Loader, Lakeflow Connect, and other ingestion methods

> *Prioritize between Auto Loader, Lakeflow Connect (standard and managed connectors), partner connectors, and other ingestion methods based on technical requirements such as data volume, ingestion frequency, data types, and governance needs with Unity Catalog.*

Situation	Best method
Millions of files, continuous, evolving schema	Auto Loader (`cloudFiles`)
Modest periodic batch of files, SQL-friendly	COPY INTO
Salesforce / Workday / SQL Server / SaaS app	Lakeflow Connect managed connector
Cloud storage / stream, standard setup	Lakeflow Connect standard connector
External database, no connector	JDBC in a notebook + Lakeflow Job
Web API	REST client in a notebook + Lakeflow Job

> **Exam tip:** prioritize by data volume, ingestion frequency, data types, and governance needs. Prefer managed connectors when they exist (least code, governed). Drop to code (JDBC/REST) only when nothing off-the-shelf fits.

Subdomain 2.7 — Ingest semi-structured and unstructured data

> *Ingest semi-structured and unstructured data (for example, JSON and nested data) via Lakeflow Connect and other managed connectors into Unity-Catalog–governed Delta tables.*

```
from pyspark.sql.functions import col, explode
# nested struct: reach fields with dot notation
df.select(col("customer.name"), col("customer.address.city"))
# array column: one row per element
df.select("order_id", explode(col("items")).alias("item"))
```

JSON with nested structs/arrays reads into struct / array columns. Flatten structs with dot-notation, expand arrays with `explode`. Auto Loader and managed connectors both land this into governed Delta tables.

Extracting JSON from a string / Variant column (Databricks SQL)

When a column holds raw JSON as a string (or a `VARIANT`), you pull fields out with the colon operator `:` — not a function:

```
-- raw_json holds a JSON string; extract fields with the colon operator
SELECT raw_json:user.id          AS user_id,     -- nested field via dot after colon
          raw_json:items[0].sku AS first_sku      -- array index too
FROM events;
```

> **Trap — the JSON operator:** Databricks SQL uses the colon `col:field.subfield` for JSON/Variant extraction. The functional equivalent is `get_json_object(col, '$.field')`. There is no `extract_json()` function — that's the invented distractor. So: colon notation, or `get_json_object`, never `extract_json`.

VARIANT type (recognize): VARIANT is Databricks' native type for semi-structured data — flexible, schema-less fields you extract without predefining a schema. Read it with the same colon operator and cast with `::TYPE`: `SELECT event_data:user.id::STRING FROM t`. Build a VARIANT from a JSON string with `parse_json(raw_json)`. So: VARIANT = the flexible semi-structured column type; `:` extracts, `::TYPE` casts.

Trap — Databricks audit-log delivery (official sample Q): audit logs are delivered as JSON, typically under ~15 minutes, and new deliveries can overwrite existing files. Not CSV, not Parquet, not weekly — remember JSON + can-overwrite when a question describes consuming Databricks audit logs from a bucket.

Code You Should Know Cold (annotated)

```
# 1) Read a CSV with a header and inferred types
df = (spark.read.format("csv")
      .option("header", "true")          # first line is column names
      .option("inferSchema", "true")     # guess types (dev only; use schema in prod)
      .load("/path/to/data"))

# 2) Read a Delta table (by path or, better, by UC name)
df = spark.read.format("delta").load("/path/to/delta_table")
df = spark.read.table("main.silver.orders")

# 3) Bridge to SQL with a temp view
df.createOrReplaceTempView("my_table")
result = spark.sql("SELECT * FROM my_table WHERE status = 'active'")

# 4) Incremental ingestion with Auto Loader
(spark.readStream.format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", "/chk/schema")
   .load("/landing/events")
 .writeStream
   .option("checkpointLocation", "/chk/progress")
   .trigger(availableNow=True)
   .toTable("main.bronze.events"))
```

Worked Examples & Field Notes

Ingestion is where "which tool?" questions live. The trick is to hear the scenario and immediately map its keywords to a method. Let's build that reflex with full examples.

Walkthrough: the same data, four ways in

Say new order files land in cloud storage. Depending on volume and cadence, four different tools are "correct" — and the exam will tell you which one by how it describes the situation.

The scenario says...	Reach for	Because
"millions of files, arriving continuously, schema may change"	Auto Loader (`cloudFiles`)	Scales, tracks state, evolves schema
"a few thousand new files each night, SQL team owns it"	COPY INTO	Simple, idempotent, pure SQL
"pull our Salesforce/Workday data in"		

The scenario says...	Reach for	Because
	Lakeflow Connect managed connector	Point-and-click, governed, no code
"read our on-prem Postgres, nightly"	JDBC in a notebook + a Job	No connector exists; drop to code

Easy picture: think of moving house. Auto Loader is a moving company with a fleet for when you own a mansion's worth of stuff. COPY INTO is renting a van for a few sensible trips. Lakeflow Connect is IKEA delivering flat-packed and pre-labeled. JDBC is you, a friend, and a hatchback, because nobody delivers to where you live.

Walkthrough: Auto Loader, line by line, and why each option exists

```
df = (spark.readStream.format("cloudFiles")                          # 1
       .option("cloudFiles.format", "json")                          # 2
       .option("cloudFiles.schemaLocation", "/Vol/_schema")          # 3
       .option("cloudFiles.inferColumnTypes", "true")                # 4
       .load("/Volumes/main/raw/orders/"))                           # 5

(df.writeStream
    .option("checkpointLocation", "/Vol/_chk")                       # 6
    .trigger(availableNow=True)                                      # 7
    .toTable("main.bronze.orders"))
```

(1) the outer format is always `cloudFiles` — this is the single most common trip-up. (2) the real file type goes here, not in the outer format. (3) where Auto Loader remembers the schema between runs so it can spot new columns. (4) without this everything comes in as strings. (5) the folder to watch. (6) the checkpoint — the bookmark of which files are done. (7) "process the backlog, then stop," which is how you get incremental behaviour on a schedule instead of a stream that runs forever.

The two folders people confuse: `schemaLocation` stores the schema (so evolution works); `checkpointLocation` stores progress (so files aren't re-read). Two folders, two jobs. Point them at the same place and you will have a confusing evening.

Coffee break. *Auto Loader's outer format is `cloudFiles` and the real format hides in `cloudFiles.format`. Every engineer writes `.format("json")` the first time, watches it ignore Auto Loader entirely, and mutters something unprintable. Consider this your free unprintable-mutter, pre-paid.*

Walkthrough: schema evolution actually happening

Monday, your orders have id, amount. Wednesday, the source team quietly adds currency and tells nobody (they never tell anybody). With default `addNewColumns` evolution, here is the sequence:

```
Run 1 (Mon):   schema = {id, amount}                        # fine
Run 2 (Wed):   new column 'currency' appears
               -> stream FAILS once, records new schema
               -> you (or the Job's retry) restart
               -> schema now = {id, amount, currency}       # succeeds, backfills nulls
```

That one deliberate failure is a feature, not a bug: it forces a checkpoint of the new shape so nothing silently disappears. Values that still don't fit the type land in a `_rescued_data` column — the lost-and-found box of ingestion.

Trap: "schema enforcement" and "schema evolution" are opposites people blur. Enforcement rejects data that doesn't match (protects the table). Evolution updates the schema to accept new columns (adapts the table). Auto Loader does both; know which word the question used.

Walkthrough: COPY INTO, the idempotent workhorse

```
COPY INTO main.bronze.orders
FROM '/Volumes/main/raw/orders/'
FILEFORMAT = JSON
FORMAT_OPTIONS ('inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Run it tonight, it loads 5,000 new files. Run it again by accident five minutes later — it loads zero, because it remembers what it already ingested. That idempotency is exactly why you use it instead of a plain `spark.read` of the folder (which would cheerfully re-load everything and duplicate your data).

Real talk. *The difference between COPY INTO and `spark.read.json(folder)` is the difference between a bouncer with a guest list and a bouncer who lets everyone in every night, including the same people, forever, until the table is 400% duplicates and someone pages you at 2am.*

Walkthrough: COPY INTO with schema evolution, run on a schedule

Here's the pattern a SQL team would schedule nightly. It loads only new files, and quietly absorbs a new column if the source adds one. Wrap that one statement in a SQL task inside a Lakeflow Job on a nightly schedule and you have a complete, idempotent ingestion pipeline with zero streaming machinery. Run it twice by accident and it loads the new files once — the second run finds nothing new and does nothing.

Exam tip: if a scenario stresses "SQL team," "simple," "periodic batch," and "into a Unity Catalog table," it's pointing at `COPY INTO`, not Auto Loader. Save Auto Loader for high volume, continuous arrival, or heavy schema evolution.

One-Minute Recap

Read pattern: `spark.read.format(f).option(k,v).schema(s).load(path)` (or `.table(name)`). Formats: CSV (needs header/inferSchema or explicit schema; inferSchema = extra scan), JSON (newline-delimited by default → `multiLine=true` for pretty/array), Parquet (self-describing, no infer), Delta (Parquet + `_delta_log` = ACID/time travel; default table format). Batch vs stream: `spark.read` = finite snapshot, re-reads each run; `spark.readStream` = incremental, tracks progress via a `checkpointLocation`; writes via `writeStream...start()/toTable()`. Auto Loader = `format("cloudFiles")`, real type in `cloudFiles.format`; needs `schemaLocation` (inferred/evolving schema, `_rescued_data` for mismatches, default `addNewColumns` evolution) and `checkpointLocation` (progress); `trigger(availableNow=True)` = catch up then stop; directory-listing vs file-notification detection. COPY INTO = SQL idempotent batch; `force=true` re-ingests (duplicates). DataFrame API: transformations (select/filter/withColumn) are lazy; actions (show/count/collect/write) execute; same API for batch & stream. Spark SQL: `createOrReplaceTempView` (session-scoped) bridges to `spark.sql(...)`; persistent governed views via `CREATE VIEW`. Notebooks: `%sql/%python/%md/%run`, `dbutils.widgets` to parameterize, `%run` (inline) vs `dbutils.notebook.run` (separate parameterized run). Databricks Connect: local IDE, remote cluster compute. Lakeflow Connect: standard (storage/stream) vs managed connectors (Salesforce/Workday/SQL Server) → UC; Connection = source auth, pipeline = destination. JDBC=DBs, REST=APIs, both via Jobs. Semi-structured: dot-notation + `explode`; SQL colon operator `/` `get_json_object` (never `extract_json`); VARIANT + `parse_json`.

DOMAIN 3

Data Transformation and Modeling

22% of the exam · roughly 10 questions

This is the heart of the exam and the work you will do most as a data engineer: taking raw bronze data and shaping it, step by careful step, into clean silver and business-ready gold. We start with the PySpark API model itself, then move through cleaning, joining, reshaping, deduplicating, aggregating, tuning, and finally the gold objects that analysts consume. Expect the most code here.

Weak-area focus (from your practice quiz): this section now folds in the patterns you missed most — the DataFrame method vs column function distinction, the window + row_number dedup pattern vs its wrong alternatives, broadcast threshold -1 & in-memory size estimation, driver vs executor memory, and CHECK constraint vs DLT expectation. Look for the "Missed-question focus" callouts.

The Medallion Architecture (the mental model for this whole domain)

Everything in Domain 3 sits inside the medallion (bronze / silver / gold) pattern. Data flows left to right, getting cleaner and more business-ready at each hop. Nearly every question in this domain is secretly asking "which layer am I in, and what operation belongs here?"

Easy picture: think of an oil refinery. Bronze is crude straight out of the ground (raw, dirty, exactly as it arrived). Silver is refined fuel (cleaned, typed, deduplicated, joined). Gold is the branded product on the shelf (aggregated, business-level tables and metrics that BI and analysts actually consume).

Layer	Contains	Typical operations	Write mode	Consumers
Bronze	Raw ingested data, append-only, schema as-is + ingest metadata	Ingest (Auto Loader / COPY INTO), keep source columns, add `_ingest_ts`, `_source_file`	Append	Data engineers
Silver	Cleaned, conformed, deduplicated, joined, validated	Null handling, type casting, joins, dedup, filters, quality checks	Append / MERGE	Engineers, DS
Gold	Aggregated business-level tables & metrics	GROUP BY aggregations, materialized views, dimensional models	Overwrite / refresh	BI, analysts, exec dashboards

Exam tip: "Clean raw data and write to a new table for downstream use" = bronze → silver. "Build an aggregated table for a BI dashboard" = silver → gold. Raw ingestion into the first landing table = → bronze. If a question describes an operation, first place it in a layer; the right answer almost always respects the layer's job.

The contrast people get wrong — why not skip layers? You could aggregate straight from bronze, but you'd repeat cleaning logic everywhere, lose a reusable clean dataset, and have no auditable "single source of truth" in silver. The layers exist so cleaning happens once and many gold tables reuse the same trusted silver.

Foundation — DataFrame Methods vs Column Functions

Before any subdomain, fix the one distinction that quietly causes wrong-API answers (e.g. picking `addColumn` or `select(F.distinct)`): DataFrame methods reshape the whole table; column functions compute a single column and are passed into a method.

	DataFrame method	Column function (pyspark.sql.functions)
Acts on	The whole DataFrame (a transformation)	One column expression
Called as	`df.method(...)`	`F.func(col)` inside a method
Returns	A new DataFrame	A Column object
Examples	`withColumn`, `withColumnRenamed`, `select`, `filter`, `distinct`, `dropDuplicates`, `groupBy`, `join`, `union`, `summary`	`col`, `when`, `sum`, `avg`, `explode`, `row_number`, `lower`, `split`

Easy picture: methods are verbs the table understands ("table, add a column", "table, remove duplicates"). Column functions are ingredients you hand to those verbs ("add a column made of this `when/otherwise`"). You never tell the table "distinct this column" — distinct is something the whole table does to itself.

```
from pyspark.sql.functions import when, col

# adding a conditional column
# WRONG: df.addColumn("flag", when(col("status")=="active","A"))       # addColumn does NOT exist
# RIGHT:
df.withColumn("flag", when(col("status")=="active","A").otherwise("B"))

# deduplicating rows
# WRONG: df.select(F.distinct("*"))       # distinct is NOT a column function
# RIGHT:
df.distinct()                   # method: dedupe on ALL columns
df.dropDuplicates(["id"])       # method: dedupe on a key subset
```

Missed-question focus — retire these non-existent names: there is no `addColumn` (use `withColumn`), no `renameColumn` (use `withColumnRenamed`), and no `F.distinct` (use the `df.distinct()` method). When unsure, ask: am I reshaping the table (method) or computing a column value (function)? That one question routes you correctly.

Subdomain 3.1 — Implement data cleaning

> *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

Read a bronze table, fix the mess, write a silver table. Three jobs the exam cares about: handle nulls, standardize data types, and persist to a new table. Do it in PySpark or SQL — you should read both fluently.

Reading the bronze table

```
# PySpark - three equivalent ways to read a managed/UC table
bronze = spark.read.table("main.bronze.orders")
```

```python
bronze = spark.table("main.bronze.orders")
bronze = spark.read.format("delta").load("/Volumes/main/bronze/orders")      # by path
```

```sql
-- SQL: just reference it by its three-level name
SELECT * FROM main.bronze.orders;
```

Handling nulls

```python
from pyspark.sql.functions import col, coalesce, lit

silver = (bronze
    .na.drop(subset=["order_id"])                    # drop rows where the KEY is null
    .na.drop(how="all")                              # drop rows where ALL cols are null
    .na.drop(thresh=3)                               # drop rows with fewer than 3 non-nulls
    .na.fill({"country": "UNKNOWN", "qty": 0})        # fill defaults per column
    .withColumn("amount", coalesce(col("amount"), lit(0.0))))      # coalesce fallback
```

```sql
-- SQL equivalents
CREATE OR REPLACE TABLE main.silver.orders AS
SELECT
   order_id,
   COALESCE(country, 'UNKNOWN')      AS country,    -- fill null
   COALESCE(amount, 0.0)             AS amount,
   NULLIF(status, '')               AS status       -- turn '' into NULL
FROM main.bronze.orders
WHERE order_id IS NOT NULL;                          -- drop null keys
```

Need	PySpark	SQL
Drop null rows	`df.na.drop(subset=[...])` / `.dropna()`	`WHERE col IS NOT NULL`
Fill nulls	`df.na.fill(value_or_dict)`	`COALESCE(col, default)`
Replace specific values	`df.na.replace(["N/A"], [None])`	`CASE WHEN` / `NULLIF`
First non-null of several	`coalesce(c1, c2, c3)`	`COALESCE(c1, c2, c3)`

> **Trap:** `na.drop` / `na.fill` only act on the columns whose type matches the fill value unless you scope with `subset`. `df.na.fill(0)` fills numeric columns only; `df.na.fill("X")` fills string columns only. Also, an empty string `''` is not null — use `NULLIF(col,'')` or a filter to treat blanks as missing.

Standardizing data types

```python
from pyspark.sql.functions import col, to_date, to_timestamp, lower, trim, regexp_replace

silver = (bronze
      .withColumn("amount",    col("amount").cast("double"))
      .withColumn("qty",       col("qty").cast("int"))
      .withColumn("order_date", to_date(col("order_ts"), "yyyy-MM-dd"))
      .withColumn("created_at", to_timestamp(col("created_str")))
      .withColumn("email",     lower(trim(col("email"))))            # normalize text
      .withColumn("phone",     regexp_replace(col("phone"), "[^0-9]", "")))
```

Easy picture: casting is relabeling boxes in a warehouse — a column of numbers stored as text ("42") is a box mislabeled "clothes"; `cast("int")` puts the right label on so downstream math, sorting and joins can trust the contents.

Trap — casting failures are silent: in Spark, casting a value that can't convert (e.g. `"abc".cast("int")`) returns NULL, it does not raise an error. So a bad cast can quietly create nulls. Validate after casting (count nulls before vs after) or you'll lose data without noticing.

Exam tip: `cast()` and `CAST(col AS type)` are the standard type-standardization tools. Know the common target types: `int`, `bigint`, `double`, `decimal(p,s)`, `string`, `boolean`, `date`, `timestamp`. Use decimal not double for money if exactness matters.

Writing the silver table

```
# overwrite the whole table
silver.write.mode("overwrite").saveAsTable("main.silver.orders")

# append (incremental loads)
silver.write.mode("append").saveAsTable("main.silver.orders")

# overwrite but allow the schema to change
(silver.write.mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable("main.silver.orders"))

# evolve schema on append (add new columns)
(silver.write.mode("append")
        .option("mergeSchema", "true")
        .saveAsTable("main.silver.orders"))
```

Write mode	Behavior if table exists
append	Adds rows to existing data
overwrite	Replaces all existing data (keeps table history via Delta)
error / errorifexists (default)	Fails if the table already exists
ignore	Does nothing if the table already exists

Exam tip: for upserts (insert new rows, update matching ones) you use `MERGE`, not a write mode — write modes can't do "update if exists". overwrite still keeps Delta history, so it's recoverable via time travel; it is not the same as dropping and recreating.

The contrast people get wrong — overwrite vs append for reruns: if a daily job might run twice, append duplicates the day's data; overwrite (or a MERGE keyed on a business key) makes the job idempotent (safe to re-run). Idempotency is a favorite silver-layer theme.

Subdomain 3.2 — Combine DataFrames with operations

> *Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.*

Operation	What it does	Keeps
Inner join	Match rows on key in both sides	Only matched rows
Left (outer) join	All left rows + matches; nulls where no match	All left rows
Right (outer) join	Mirror of left	All right rows
Full outer join	All rows from both, nulls where unmatched	Everything
Left semi join	Left rows that HAVE a match (no right columns added)	Filtered left, like an IN
Left anti join	Left rows with NO match (like NOT IN)	Unmatched left only
Cross join	Cartesian product, every row × every row	N×M rows (dangerous)
Broadcast join	Ships a small table to every executor to avoid a shuffle	Same result as the join type, but faster
union / unionAll	Stack rows of two same-schema DataFrames	All rows (Spark's union = SQL UNION ALL, keeps dups)

```
from pyspark.sql.functions import broadcast

# inner join on one key (string form dedupes the join column)
orders.join(customers, "customer_id", "inner")

# join on MULTIPLE keys
orders.join(cust, on=["customer_id", "region"], how="left")

# join on an EXPRESSION (keeps both key columns - watch for duplicates)
orders.join(cust, orders.cust_id == cust.id, "left")

# broadcast the small dim table (forces a broadcast, skips the shuffle)
orders.join(broadcast(small_dim), "product_id")

# semi / anti - filtering joins
orders.join(active_cust, "customer_id", "left_semi")   # orders that HAVE an active customer
orders.join(active_cust, "customer_id", "left_anti")   # orders with NO active customer

# stacking rows
jan.union(feb)              # keeps duplicates (like SQL UNION ALL)
jan.union(feb).distinct()   # dedupe = SQL UNION behavior
jan.unionByName(feb)        # match columns by NAME, not position
```

Trap 1 — union keeps duplicates: In Spark, `DataFrame.union()` and `unionAll()` are identical — both keep duplicates. This is the opposite of SQL, where `UNION` dedupes and `UNION ALL` keeps dups. To dedupe in PySpark, chain `.distinct()`.

Trap 2 — union matches by position: union lines up columns by ordinal position, not name. If two DataFrames have the same columns in a different order, you'll silently mix data into the wrong columns. Use `unionByName()` when schemas might differ in order (and `allowMissingColumns=True` if column sets differ).

The contrast people get wrong — broadcast vs shuffle join: a normal join shuffles both tables across the network by key (expensive). A broadcast join copies the small table to every executor so the big table never moves. Rule: one table is small (fits in memory, under `autoBroadcastJoinThreshold`, default 10 MB) → broadcast. Both huge

→ regular shuffle join. Spark auto-broadcasts tables under the threshold; `broadcast()` forces it when Spark can't estimate the size (e.g. after complex transforms).

Trap 3 — cross join explosion: a cross join of a 1M-row table with a 1M-row table produces 1 trillion rows. Spark even requires `crossJoin()` or `spark.sql.crossJoin.enabled` to prevent accidental Cartesian products. An unintended cross join usually comes from a join condition that's always true or a missing key.

Exam tip: a star-schema join of one giant fact table to several small dimension tables → broadcast the dimensions. Need "rows in A that exist in B" without pulling B's columns → left semi. Need "rows in A missing from B" (e.g. new records) → left anti.

Subdomain 3.3 — Manipulate columns, rows, and table structures

> *Manipulate columns, rows, and table structures by adding, dropping, splitting, renaming column names, applying filters, and exploding arrays.*

Task	PySpark	SQL
Add / compute a column	`df.withColumn("tax", col("amt")*0.2)`	`SELECT amt*0.2 AS tax`
Rename a column	`df.withColumnRenamed("old","new")`	`SELECT old AS new` / `ALTER TABLE ... RENAME COLUMN`
Drop columns	`df.drop("c1","c2")`	`ALTER TABLE ... DROP COLUMN`
Select / reorder	`df.select("id","name")`	`SELECT id, name`
Filter rows	`df.filter(col("qty") > 0)` / `.where(...)`	`WHERE qty > 0`
Split a string column	`split(col("full_name"), " ")`	`split(full_name, ' ')`
Explode an array into rows	`df.select(explode(col("items")))`	`LATERAL VIEW explode(items)`

Adding, renaming, dropping

```
from pyspark.sql.functions import col, when, concat_ws

df = (df
   .withColumn("total", col("qty") * col("price"))
   .withColumn("tier", when(col("total") > 1000, "gold").otherwise("standard"))
   .withColumnRenamed("cust_nm", "customer_name")
   .drop("_corrupt_record", "raw_line"))

# add/rename multiple at once (Spark 3.4+)
df = df.withColumnsRenamed({"a": "alpha", "b": "beta"})
```

Filtering rows

```
df.filter((col("qty") > 0) & (col("country").isin("US","EU")))
df.where("qty > 0 AND country IN ('US','EU')")          # SQL-string form
df.filter(col("email").isNotNull() & col("email").contains("@"))
```

Trap — Python vs Spark operators: in PySpark you must use `&`, `|`, `~` (not `and` / `or` / `not`) and wrap each condition in parentheses because `&` binds tighter than `>`. `col("a") > 0 & col("b") > 0` throws; `(col("a") > 0) & (col("b") > 0)` works.

Splitting strings & exploding arrays

```
from pyspark.sql.functions import split, explode, explode_outer, col, size

# split "first last" into two columns
df = (df.withColumn("parts", split(col("full_name"), " "))
             .withColumn("first", col("parts")[0])
             .withColumn("last",    col("parts")[1])
             .drop("parts"))

# explode: one row per array element (drops empty/null arrays)
exploded = orders.select("order_id", explode(col("items")).alias("item"))

# explode_outer keeps rows whose array is null/empty (emits a null item)
kept = orders.select("order_id", explode_outer(col("items")).alias("item"))
```

Easy picture: `explode` unpacks a moving box. One order row whose items column is a box holding [shirt, hat, mug] becomes three rows — one per item — so you can count and aggregate individual products. `split` is the reverse-ish: it takes one string and slices it into an array you can then index or explode.

Exam tip: `explode` drops rows whose array is null or empty; `explode_outer` keeps them. Reach nested struct fields with dot notation (`col("address.city")`). To go from many rows back to one array use `collect_list` / `collect_set` in a groupBy.

Subdomain 3.4 — Perform deduplication and aggregate operations

> *Perform data deduplication operations and aggregate operations on DataFrames, such as count, approximate count distinct, and mean, summary.*

Deduplication

```
# 1) simple: drop exact / key-based duplicates (keeps an ARBITRARY row per key)
df.dropDuplicates(["order_id"])
df.distinct()                           # dedupe on ALL columns

# 2) keep the LATEST record per key (the real-world pattern)
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

w = Window.partitionBy("order_id").orderBy(col("updated_at").desc())
```

```
latest = (df.withColumn("rn", row_number().over(w))
            .filter(col("rn") == 1).drop("rn"))
```

Function	Rows kept per key
`row_number()`	Exactly one (ties broken arbitrarily) — use for dedup
`rank()`	Ties share a rank, leaves gaps (1,1,3)
`dense_rank()`	Ties share a rank, no gaps (1,1,2)

Exam tip: plain `dropDuplicates` keeps an arbitrary row per key. To keep a specific one (newest, highest value), use a window + `row_number()` ordered appropriately and filter to `rn == 1`. This is the canonical "keep latest version of each record" pattern in silver.

Trap — dedup on the wrong columns: `dropDuplicates()` with no arguments dedupes on every column, so two rows differing only in a timestamp are both kept. Pass the business key subset (`["order_id"]`) to dedupe on identity, not the whole row.

Why the two tempting alternatives are wrong

Approach	Keeps all columns?	Deterministic?	Verdict
Window + row_number()==1	Yes	Yes	Correct pattern
orderBy + dropDuplicates	Yes	No (sort not respected)	Wrong — arbitrary row
groupBy + agg(max)	No (loses other cols)	Yes for the value	Wrong — can't return full row

Missed-question focus — window vs orderBy + dropDuplicates: sorting then calling `dropDuplicates` does not keep the latest. `dropDuplicates` shuffles by key and retains an arbitrary row per key — the earlier global sort is discarded, so the result is non-deterministic. Only a window ties the ordering to the grouping.

Missed-question focus — window vs groupBy + agg: `groupBy(key).agg(max("ts"))` returns only the key and the aggregated value — every other column is dropped. It answers "what is the max timestamp per key?" but cannot hand back the full winning row. When you need the whole record of the latest version, use the window.

Exam trigger phrases: "keep the latest/most recent record per ...", "retain all columns of the winning row", "deterministic deduplication" → window + `row_number()==1`. Use `row_number()` (not `rank()` / `dense_rank()`) when you need exactly one row even under ties.

Aggregation

Function	Meaning	Note
`count("*")`	Row count	Counts all rows incl. nulls
`count("col")`	Non-null count of a column	Ignores nulls
`countDistinct("col")`	Exact distinct count	Accurate, but shuffles heavily
`approx_count_distinct("col")`	Estimated distinct (HyperLogLog)	Much faster/less memory on huge data; ~small error

Function	Meaning	Note
`avg()` / `mean()`	Average	Identical functions
sum, min, max, stddev	Standard aggregates	—
`df.summary()`	count, mean, stddev, min, quartiles, max	Full statistical profile
`df.describe()`	count, mean, stddev, min, max	Lighter than summary (no quartiles)

```
from pyspark.sql.functions import sum, avg, approx_count_distinct, count, countDistinct

gold = (silver.groupBy("region")
    .agg(sum("amount").alias("revenue"),
         avg("amount").alias("avg_order"),
         count("*").alias("orders"),
         approx_count_distinct("customer_id").alias("customers")))

# quick profiling of a DataFrame
df.summary().show()                                    # all stats
df.summary("count", "mean", "min", "max").show() # chosen stats
df.describe("amount").show()
```

The contrast people get wrong — count vs approx_count_distinct: exact `countDistinct` must shuffle and track every unique value (heavy on billions of rows). `approx_count_distinct` uses HyperLogLog sketches — tiny memory, ~2% default error, huge speedup. Rule: dashboards/monitoring on massive data where "close enough" is fine → approx. Billing/audit exact numbers → exact count. You can tighten the error with the second arg (`rsd`), at higher cost.

Trap — count("*") vs count(col): `count("*")` counts every row; `count("col")` counts only rows where col is not null. If a question asks "how many orders have a non-null email", that's `count("email")` , not `count("*")` .

Window functions beyond dedup (the exam uses these a lot)

Windows compute a value per row over a group, without collapsing rows (unlike `groupBy`). Same `Window.partitionBy(...).orderBy(...)` skeleton, different function:

Function	Gives you
`row_number()`	Unique 1,2,3 per group — the dedup / top-N tool (keep `rn==1`)
`rank()` / `dense_rank()`	Ranking with ties: rank leaves gaps (1,1,3), dense_rank doesn't (1,1,2)
`lag()` / `lead()`	Previous / next row's value — e.g. days between consecutive orders (`datediff` vs `lag("order_date")`)
`ntile(n)`	Split ordered rows into n equal buckets (quartiles = `ntile(4)`)
Running total / moving avg	`sum(...).over(w.rowsBetween(Window.unboundedPreceding, Window.currentRow))`

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, lag, ntile, datediff, col
w = Window.partitionBy("store_id").orderBy("sale_date")
```

```
df.withColumn("running_total",
     sum("daily_sales").over(w.rowsBetween(Window.unboundedPreceding, Window.currentRow)))
```

Exam tip: "difference between consecutive rows" → `lag`. "equal-sized buckets / quartiles" → `ntile(n)`. "running total / moving average" → `sum/avg over a rowsBetween frame`. "one row per key" → `row_number()==1`.

Subdomain 3.5 — Understand the basic tuning parameters

> *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

Parameter	Controls	Default	Rule of thumb
`spark.sql.shuffle.partitions`	Partitions after a shuffle (joins, groupBy, window)	200	Too high on small data = tiny wasteful tasks; too low on big data = spill/skew. Match to data size
`spark.default.parallelism`	Default partition count for RDD ops (non-SQL)	= total executor cores	Usually leave at cores; affects RDD/parallelize, not DataFrame shuffles
`spark.executor.memory`	RAM per executor (does the work)	cluster-dependent	Raise to fix executor OOM / spill
`spark.driver.memory`	RAM for the driver (collects results, plans, broadcasts)	cluster-dependent	Raise if `collect()` / `toPandas()` /broadcast OOMs the driver
`spark.sql.autoBroadcastJoinThreshold`	Max table size auto-broadcast in a join	10 MB	Raise to broadcast bigger dims; set -1 to disable auto-broadcast

```
spark.conf.set("spark.sql.shuffle.partitions", 64)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)           # 50MB
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)                         # disable

# read current value
print(spark.conf.get("spark.sql.shuffle.partitions"))
```

Easy picture: `shuffle.partitions` is how many lanes you open at a toll plaza after a merge. 200 lanes for 5 cars wastes booths (tiny tasks); 2 lanes for a million cars causes a jam (spill). Match lanes to traffic.

Broadcast threshold: the -1 rule and what size Spark actually measures

Value of autoBroadcastJoinThreshold	Effect
-1	Disables auto-broadcast completely → forces a shuffle/sort-merge join
0	Effectively disables (nothing is ≤ 0 bytes)
10485760 (default 10 MB)	Tables estimated ≤ 10 MB are broadcast
positive N	Tables ≤ N broadcast; larger ones shuffle

Missed-question focus — only -1 guarantees "no broadcast": if an 8 MB table keeps auto-broadcasting and failing, setting the threshold to 10 MB does not help — 10 MB is still above 8 MB, so it still broadcasts. A positive number only moves the cutoff; to guarantee no auto-broadcast (force a shuffle join) set -1. That's the "never broadcast" value.

Missed-question focus — on-disk vs in-memory size: Spark compares the threshold to the uncompressed, deserialized in-memory size, not the compressed on-disk Parquet file size. A 7 MB Parquet file can expand to 30+ MB in memory and blow past a 10 MB threshold — so it does not broadcast even though the file "looks" small. Parquet format never enables/disables broadcast by itself; only estimated in-memory size vs the threshold decides.

Easy picture: Parquet on disk is a vacuum-packed suitcase. The threshold check happens after unpacking — the clothes puff up to real volume. Judging by the flat suitcase (file size) fools you; Spark measures the unpacked pile (in-memory size).

Re-measuring performance (the part the objective names explicitly)

"Understand the basic tuning parameters and re-measure the performance" means: change one parameter, re-run, and compare — don't guess. Where to look:

What to compare	Where	Good sign
Total job / stage duration	Spark UI → Jobs / Stages	Shorter after the change
Spill (memory/disk)	Spark UI → Stage detail	Spill drops to zero
Task time distribution	Stage → task min/median/max	Max ≈ median (balanced, no skew)
Shuffle read/write bytes	Stage detail	Lower after broadcast/filtering

Exam tip: Adaptive Query Execution (AQE), on by default in modern DBR, dynamically coalesces shuffle partitions, switches join strategies, and handles skew at runtime — so hand-tuning shuffle.partitions matters most when AQE is off or for stubborn edge cases. Mentioning AQE as the "automatic" alternative to manual tuning is a common right answer.

The contrast people get wrong — driver vs executor memory: executor memory does the distributed work (raise it for spill/OOM inside stages); driver memory holds results you pull back and broadcast tables you build. A job that dies on `collect()` or a big broadcast needs more driver memory (or, better, don't collect huge data). A job spilling mid-shuffle needs more executor memory or more partitions.

Performance-aware coding (the exam's efficiency angle)

Habit	Why it's faster
Filter & select columns early	Less data to shuffle/scan (predicate & column pruning)
Prefer built-in functions over Python UDFs	Built-ins run in the JVM/Photon; Python UDFs are slow and skip Photon
`broadcast()` the small side of a join	Avoids shuffling the big table
`repartition(n)` vs `coalesce(n)`	repartition = full shuffle to n (can grow or shrink, even sizes); coalesce = shrink only, no shuffle (cheaper)

Habit	Why it's faster
`cache()` a DataFrame reused many times	Avoids recomputing the same plan repeatedly
Avoid `collect()` / `toPandas()` on big data	Pulls everything to the driver → OOM risk

Partitioning vs Liquid Clustering: classic Hive `PARTITIONED BY (col)` physically splits files by a column's values — good only for low-cardinality columns (e.g. date). Partition on a high-cardinality column (like user_id) and you get millions of tiny files (the "small files problem"). On new tables prefer Liquid Clustering (`CLUSTER BY`): it gives data skipping without that trap and its keys can change later. Exam tell: "avoid the small-files problem / change the layout later" → Liquid Clustering.

Subdomain 3.6 — Build Gold layer objects

> *Understand the difference between, and how to build, Gold layer objects such as materialized views, views, streaming tables, and tables for BI and analytics teams in Unity Catalog.*

Object	What it is	Stor-age	Freshness	Use when
Table (man-aged)	Physically stored Delta data	Stored	Static until you rewrite it	Curated data you write and control
View	Saved query, computed at read time	None	Always live (re-runs each query)	Lightweight logic, no stor-age, always current
Materialized view	Precomputed query result, stored & incrementally refreshed	Stored	Refreshed on schedule / trigger	Expensive aggregations queried often by BI
Streaming table	Delta table continuously/incrementally appended from a streaming source	Stored	Continuously up-dated	Incremental ingestion & near-real-time pipelines

Easy picture: a view is a recipe — nothing is cooked until someone orders (queries), so it's always fresh but you pay to cook every time. A materialized view is a batch of meals cooked ahead and kept warm — instant to serve, refreshed periodically. A streaming table is a conveyor belt that keeps adding new plates as ingredients arrive. A plain table is a dish already plated and sitting on the pass — fast, but only as fresh as when you last cooked it.

```
-- VIEW: logic only, recomputed each read
CREATE VIEW main.gold.active_customers AS
SELECT * FROM main.silver.customers WHERE status = 'active';

-- MATERIALIZED VIEW: stored + incrementally refreshed
CREATE MATERIALIZED VIEW main.gold.daily_revenue
AS SELECT order_date, sum(amount) AS revenue
FROM main.silver.orders GROUP BY order_date;

-- refresh it (or schedule it)
REFRESH MATERIALIZED VIEW main.gold.daily_revenue;

-- STREAMING TABLE: incremental append from a streaming source
CREATE STREAMING TABLE main.silver.orders_stream
AS SELECT * FROM STREAM read_files('/Volumes/main/raw/orders/', format => 'json');

-- plain managed TABLE built by a batch job
```

```
CREATE TABLE main.gold.region_summary AS
SELECT region, sum(amount) rev FROM main.silver.orders GROUP BY region;
```

The contrast people get wrong — view vs materialized view: a view stores no data and recomputes every time (always fresh, but slow & costly on big aggregations). A materialized view stores the result and refreshes it, so BI reads are instant — at the cost of storage and some staleness between refreshes. Rule: "expensive aggregation hit repeatedly by a dashboard" → materialized view. "Simple filter that must always be current" → view.

The contrast people get wrong — streaming table vs materialized view (the Gold-layer decision): a streaming table is append-only and low-latency — it ingests a growing stream (e.g. clickstream into bronze) but it does not recompute joins and cannot handle sources that get UPDATEs/DELETEs. A materialized view stores the result of a query (aggregations, joins) and incrementally refreshes it — and, unlike a streaming table, it works on both append-only and update/delete sources. Rule for the exam: a Gold BI aggregation / join, or a Silver source with updates/deletes → materialized view (the recommended Gold object). Continuous append-only ingestion with no joins → streaming table. A plain view is wrong for heavy Gold aggregations because it recomputes every query.

SQL syntax (memorize): in Databricks SQL a streaming table reads a source with the `STREAM()` keyword — `CREATE STREAMING TABLE t AS SELECT * FROM STREAM(source_table)`. There is no `read_stream()` SQL function — that mistake comes from confusing PySpark's `spark.readStream` with SQL. PySpark = `readStream`; SQL = `STREAM(...)`.

Exam tip: materialized views and streaming tables in UC are typically created and refreshed by Lakeflow Declarative Pipelines (formerly DLT) or Databricks SQL. All four are governed Unity Catalog objects addressed with the three-level namespace `catalog.schema.object`, so BI/analytics teams get consistent grants, lineage, and discovery.

What a Lakeflow Declarative Pipeline is (formerly Delta Live Tables / DLT)

Several gold objects above (streaming tables, materialized views) are typically built by a Lakeflow Declarative Pipeline. It's a declarative ETL framework: you declare the target tables in SQL or Python, and the pipeline works out the dependency order, does the incremental processing, and manages the checkpoints and schema locations for you. You state what the tables should contain; it handles how to build and keep them fresh. Its built-in data-quality rules are expectations (`expect` / `expect_or_drop` / `expect_or_fail`).

Declarative Pipeline vs Lakeflow Job: a Declarative Pipeline is a single, self-managing data-processing asset (it figures out dependencies and incremental logic). A Lakeflow Job is a general orchestrator that runs things in order — and it can run a pipeline as one of its tasks. Rule: "declaratively define streaming tables / materialized views with managed dependencies & quality" → Declarative Pipeline; "schedule and chain arbitrary tasks" → Lakeflow Job.

Subdomain 3.7 — Apply data quality checks and validation rules

> *Apply data quality checks and validation rules to ensure reliable Silver and Gold datasets.*

Silver and gold data must be trustworthy. Databricks gives you a spectrum from soft observability to hard enforcement.

Mechanism	How it behaves	Scope
Delta CHECK constraint	Rows violating the rule fail the write (hard enforcement)	Table-wide, every writer

Mechanism	How it behaves	Scope
NOT NULL constraint	Rejects null in a required column at write time	Table-wide
Pipeline expectations (`EXPECT`)	In Lakeflow Declarative Pipelines: warn, drop, or fail the row/pipeline	Within that pipeline
Manual validation query	Count violations, route to a quarantine table, alert	Wherever you run it

Delta constraints (hard, table-wide)

```
-- CHECK: violating writes are rejected outright
ALTER TABLE main.silver.orders
  ADD CONSTRAINT valid_amount CHECK (amount > 0);

-- NOT NULL: required columns
ALTER TABLE main.silver.orders
  ALTER COLUMN order_id SET NOT NULL;

-- inspect / drop
DESCRIBE DETAIL main.silver.orders;                 # see constraints
ALTER TABLE main.silver.orders DROP CONSTRAINT valid_amount;
```

Pipeline expectations (flexible, in-pipeline)

```
# Lakeflow Declarative Pipeline expectations (Python)
@dp.table
@dp.expect("valid_amount", "amount > 0")                        # WARN: keep row, count violations
@dp.expect_or_drop("has_id", "order_id IS NOT NULL")            # DROP bad rows, continue
@dp.expect_or_fail("positive_qty", "qty >= 0")                   # FAIL the whole pipeline
def silver_orders():
     return spark.readStream.table("main.bronze.orders")
```

```
-- SQL form in a declarative pipeline
CREATE OR REFRESH STREAMING TABLE silver_orders(
  CONSTRAINT valid_amount   EXPECT (amount > 0),
  CONSTRAINT has_id         EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT positive_qty   EXPECT (qty >= 0)      ON VIOLATION FAIL UPDATE
) AS SELECT * FROM STREAM(main.bronze.orders);
```

The quarantine pattern (manual)

```
# split good vs bad rows, keep bad ones for inspection instead of losing them
from pyspark.sql.functions import col
rule = (col("amount") > 0) & col("order_id").isNotNull()
df.filter(rule).write.mode("append").saveAsTable("main.silver.orders")
df.filter(~rule).write.mode("append").saveAsTable("main.silver.orders_quarantine")
```

> **The contrast people get wrong — expect / expect_or_drop / expect_or_fail:** plain `expect` keeps bad rows and just records the violation metric (great for observability). `expect_or_drop` removes bad rows but the pipeline continues. `expect_or_fail` halts the whole pipeline. Rule: monitor → expect; clean → drop; must-be-perfect → fail.

Missed-question focus — DLT syntax is CONSTRAINT ... EXPECT, not CHECK: inside a Declarative Pipeline (DLT) SQL definition you write `CONSTRAINT name EXPECT (condition) [ON VIOLATION DROP ROW | FAIL UPDATE]`. Plain SQL `CHECK (…)` is not supported in a DLT/streaming-table definition — CHECK is only for standard Delta tables via `ALTER TABLE … ADD CONSTRAINT … CHECK`. So: DLT/pipeline → `EXPECT`; standard Delta table → `CHECK`. And bare `EXPECT` (no `ON VIOLATION`) = monitor-only (keeps rows), `DROP ROW` = drop, `FAIL UPDATE` = fail.

Python SDP (Lakeflow Spark Declarative Pipelines) decorators — current syntax: import is from `pyspark import pipelines as dp`. Single-rule: `@dp.expect` / `@dp.expect_or_drop` / `@dp.expect_or_fail("name", "rule")`. Multiple rules at once with a dict: `@dp.expect_all({...})` (warn), `@dp.expect_all_or_drop({...})` (drop if any fails), `@dp.expect_all_or_fail({...})` (fail the update if any fails). The _all variants take a {name: constraint} dictionary.

The contrast people get wrong — CHECK constraint vs pipeline expectation: a CHECK constraint is enforced by Delta on every write to that table, by any writer, and rejects bad rows (write fails). A pipeline expectation only applies inside that pipeline and can be configured to warn/drop/fail. For a hard, table-wide guarantee that no bad data ever lands, use the constraint; for flexible in-flight handling with metrics, use expectations.

Exam tip: constraints and expectations are the two names to know. "Reject bad writes for everyone, permanently" → CHECK/NOT NULL constraint. "Track a data-quality metric and optionally drop/fail in a pipeline" → expectations. Quarantine tables are the pattern when you must keep bad rows for investigation rather than discard them.

Cross-Cutting: Idempotency & Incremental Silver Loads

A recurring silver-layer theme the exam rewards: pipelines should be idempotent (safe to re-run) and support incremental updates. The tool is `MERGE`.

```
-- upsert: THE pattern for incremental silver loads (insert new, update changed)
MERGE INTO main.silver.orders AS t
USING staged_updates AS s
   ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Exam tip: a re-run of an append job duplicates data; a MERGE keyed on the business key updates in place and inserts only new rows, so re-running is safe. This is why silver often uses MERGE rather than plain append.

Worked Examples & Field Notes

This chapter is where the exam turns into actual PySpark. The good news: a handful of patterns cover almost everything. Let's drill them with full, runnable examples and the traps that hide inside each.

Walkthrough: a complete bronze → silver clean

```
from pyspark.sql.functions import col, to_timestamp, coalesce, lit, row_number, lower, trim
from pyspark.sql.window import Window

bronze = spark.read.table("main.bronze.customers")

typed = (bronze
```

```
        .na.drop(subset=["customer_id"])                        # no key = useless row
        .withColumn("email", lower(trim(col("email"))))           # normalize text
        .withColumn("signup_ts", to_timestamp(col("signup")))     # string -> timestamp
        .withColumn("country", coalesce(col("country"), lit("UNKNOWN"))))

# keep the freshest row per customer, keeping ALL columns
w = Window.partitionBy("customer_id").orderBy(col("signup_ts").desc())
silver = (typed.withColumn("rn", row_number().over(w))
                  .filter(col("rn") == 1).drop("rn"))

silver.write.mode("overwrite").saveAsTable("main.silver.customers")
```

Every line here is an exam topic in disguise: null handling, casting, the window-dedup pattern, and a governed write. Learn this skeleton and swap the columns.

Coffee break. *Bronze data is like a teenager's bedroom: technically everything you need is in there, but you would not invite guests (analysts) in until someone has cleaned it. Silver is the room after cleaning. Gold is the staged photo you put on the listing.*

The dedup pattern, and why the tempting shortcuts betray you

Approach	Keeps all columns?	Deterministic?	Verdict
Window + row_number()==1	Yes	Yes	Correct
orderBy().dropDuplicates(["id"])	Yes	No	The sort is thrown away in the shuffle
groupBy("id").agg(max("ts"))	No	Yes	You get id + max only; every other column vanishes

> **Easy picture:** groupBy + max is asking "what's the latest timestamp per customer?" and getting a sticky note with just a date on it. The window is asking "give me the entire most-recent record for each customer" and getting the whole file. The exam wants the whole file.

Real talk. *dropDuplicates after a sort is the data-engineering equivalent of alphabetizing your bookshelf and then letting a toddler "help" carry the books to a new room. The order you carefully made does not survive the trip.*

Walkthrough: the silent-null cast trap

```
df.withColumn("age", col("age").cast("int"))
```

Looks harmless. But if age contains "unknown" for some rows, Spark does not raise an error — it quietly turns those into `NULL`. You can lose data and never see a red message. The professional move is to check before and after:

```
before = df.filter(col("age").isNotNull()).count()
after  = df.withColumn("age", col("age").cast("int")).filter(col("age").isNotNull()).count()
print(before - after, "values silently became null on cast")
```

Coffee break. *Spark casting is the friend who says "yeah, I handled it" and technically did, in the sense that the problem is now a bunch of quiet nulls instead of an error. Trust, but count().*

Aggregation subtleties the exam loves

```
count("*")                      # every row, nulls included
count("email")                  # only rows where email is NOT null — different number!
approx_count_distinct("uid")    # HyperLogLog estimate: ~2% off, but fast on billions
avg("x")                        # identical to mean("x")
```

Exam tip: "fast distinct count on a huge table, small error is fine" → `approx_count_distinct`. "How many rows have a non-null email" → `count("email")`, not `count("*")`. These two swaps show up constantly.

Gold objects, chosen correctly

You need...	Build a...
Always-current logic, no storage, cheap	View (recomputes each read)
Expensive aggregation a dashboard hits all day	Materialized view (stored, refreshed)
Continuously arriving data, incrementally appended	Streaming table
Curated data you write and control	Managed table

Real talk. *A plain view recomputing a billion-row aggregation on every dashboard refresh is why some BI dashboards take ninety seconds to load and everyone quietly stops using them. Materialized views exist so your dashboard loads before the analyst loses faith in you.*

Walkthrough: a complete silver → gold aggregation

```
from pyspark.sql.functions import sum, avg, count, approx_count_distinct, col

silver = spark.read.table("main.silver.orders")

gold = (silver
       .filter(col("status") == "completed")              # only real revenue
       .groupBy("region", "order_date")                     # the dimensions
       .agg(sum("amount").alias("revenue"),                 # the measures
              avg("amount").alias("avg_order"),
              count("*").alias("orders"),
              approx_count_distinct("customer_id").alias("customers")))

gold.write.mode("overwrite").saveAsTable("main.gold.daily_region_revenue")
```

Exam tip: notice the split — group by the "per what" columns (region, date), aggregate the numbers inside functions. Putting the measure in `groupBy`, or calling `sum()` with no argument, is the classic aggregation mistake.

One-Minute Recap

API model: methods reshape the table (`withColumn` , `withColumnRenamed` , `distinct` , `dropDuplicates` , `groupBy`); functions compute a column (`when` , `row_number` , `sum`) — no `addColumn` , no `F.distinct` . Medallion: bronze (raw) → silver (clean, typed, deduped, joined, validated) → gold (aggregated for BI). Cleaning: read with `spark.read.table` ; `na.drop` / `na.fill` / `coalesce` for nulls (`''` ≠ null), `cast()` for types (bad casts → silent NULL), write with `overwrite` / `append` / `error` / `ignore` ; upserts use `MERGE` . Joins: inner/left/right/full/semi/anti/cross; `broadcast()` a small table to skip a shuffle; Spark union = unionAll = keeps dups (add `.distinct()`) and matches by position (`unionByName` otherwise). Structure: `withColumn` / `withColumnRenamed` / `drop` / `filter` (use `&|~` + parens)/ `split` ; `explode` = array→rows (drops empty; `explode_outer` keeps). Dedup: `dropDuplicates([key])` keeps arbitrary row; window `row_number()` keeps a chosen one (newest). Aggregation: `count("*")` all rows vs `count(col)` non-null; `approx_count_distinct` = fast HLL estimate; `avg=mean` ; `summary()` / `describe()` profile. Tuning: `shuffle.partitions` (200), `default.parallelism` , executor/driver.memory (executor=stage work, driver=collect/broadcast), `autoBroadcastJoinThreshold` (10 MB; -1 disables — a positive N only moves the cutoff; Spark measures uncompressed in-memory size, not the compressed file); re-measure in Spark UI; AQE auto-coalesces/handles skew. Gold objects: view=live recompute, materialized view=stored+refreshed (fast BI), streaming table=incremental append, table=static stored — all UC-governed `catalog.schema.object` . Quality: CHECK/NOT NULL constraints reject bad writes table-wide; pipeline `expect` (warn) / `expect_or_drop` (drop) / `expect_or_fail` (halt); quarantine pattern keeps bad rows. Idempotency: MERGE on business key = safe re-runs.

DOMAIN 4

Working with Lakeflow Jobs

16% of the exam · roughly 7 questions

A pipeline is only useful if it runs reliably, in the right order, at the right time. Lakeflow Jobs is the orchestrator that makes that happen. This chapter covers how tasks form a dependency graph, how to add control flow like retries and branching, and — most importantly for the exam — how to choose the right trigger so a pipeline runs when the data is actually ready.

What Lakeflow Jobs Is

Lakeflow Jobs (formerly "Databricks Jobs" / "Workflows") is the platform's native orchestrator: it runs your notebooks, SQL, pipelines and scripts on a schedule or a trigger, in a defined order, with retries and alerting. A job is the whole workflow; a task is one step; dependencies between tasks form a DAG (directed acyclic graph).

> **Easy picture:** a job is a recipe with numbered steps. Each task is a step ("chop onions", "fry", "plate"). The DAG is the arrows saying which steps must finish before the next starts. You can't plate before frying — the arrows enforce that.

> **Exam tip:** Lakeflow Jobs orchestrates many task types together. A Lakeflow Declarative Pipeline (formerly DLT) is a single data-processing asset that a job can run as one task. Don't confuse the orchestrator (Jobs) with the pipeline it orchestrates.

Subdomain 4.1 — Implement control flows using Lakeflow Jobs

> *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration.*

Control feature	What it does
Retries	Re-run a failed task N times with a wait interval before giving up
If/else condition task	Branches the DAG on a boolean expression (e.g. a task value)
For-each task	Loops a task over a list of inputs, optionally in parallel
Run-if dependency	Controls when a task runs relative to upstream status: all succeeded, at least one succeeded, none failed, all done, etc.
Duration threshold / timeout	Warn or fail if a task/job runs too long

Retries

> **Easy picture:** retries are hitting "redial" when a call drops. A transient network blip or a briefly-locked table often succeeds on the second try — retries stop a one-off hiccup from failing the whole pipeline.

> **Exam tip:** retries are ideal for transient/flaky failures (network, momentary unavailability). They will not fix a deterministic bug (bad SQL, missing column) — those fail every retry. Set a sensible max-retries and retry interval.

Conditional branching (If/else)

```
# Task A sets a value; a condition task branches on it
# Task A (Python):
row_count = df.count()
dbutils.jobs.taskValues.set(key="rows", value=row_count)

# If/else condition task expression:
{{tasks.check.values.rows}} > 0
# true     -> run load_gold
# false -> run send_no_data_alert
```

Looping (For-each)

A For-each task runs a nested task once per element of an input array (e.g. a list of regions or file paths), optionally with a concurrency limit so iterations run in parallel.

> **The contrast people get wrong — orchestration branching vs code branching:** if/else condition tasks and for-each loops live in the job DAG, so branches show up in the run graph and are visible/monitorable. Putting the same logic as if statements inside one giant notebook hides it from the orchestrator. Rule: when the exam says "branch/loop using Lakeflow Jobs", it wants the condition/for-each task, not in-notebook code.

Subdomain 4.2 — Configure common tasks and their dependencies

> *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph.*

Common task types

Task type	Runs	Typical use
Notebook	A Databricks notebook (Python/SQL/Scala)	Transformations, general logic
SQL	A SQL query, alert, or file on a SQL warehouse	Analytics, checks, refreshes
Dashboard	Refreshes a Lakeview/DBSQL dashboard	Push fresh BI after data loads
Pipeline	A Lakeflow Declarative Pipeline	Declarative ETL as one step
Python script / wheel, JAR, dbt, Spark Submit	Code artifacts	Custom / packaged workloads
Run Job	Another job	Compose modular jobs

Building the dependency graph

Each task declares which tasks it depends on. A task only starts once all its upstream dependencies succeed. Independent tasks with no shared dependency run in parallel.

```
# Job DAG (conceptual):
ingest_bronze
       |
clean_silver ──────────┐
       |               |
build_gold      quality_checks       # these two run in parallel
       |               |
       └──── refresh_dashboard ─────┘        # waits for BOTH
```

Exam tip: tasks with the same single upstream parent and no link to each other run concurrently. A downstream task with two parents waits until both finish. This is how the DAG expresses "fan-out" and "fan-in".

Compute & parameters

Tasks can share a job cluster (cheaper jobs compute, created for the run and torn down) or use serverless. Pass values with job/task parameters and read them via task values (`dbutils.jobs.taskValues.set/get`) to hand results from one task to the next.

The contrast people get wrong — job cluster vs all-purpose: point production job tasks at a job cluster (or serverless), not an always-on all-purpose cluster. Job clusters use the cheaper jobs DBU rate and exist only during the run. Reusing one job cluster across tasks in the same run avoids repeated startup cost.

Subdomain 4.3 — Implement job schedules using Lakeflow Jobs

> *Implement job schedules using Lakeflow Jobs with an understanding of trigger types (scheduled, file arrival, and table update).*

Trigger	Fires when	Nature
Scheduled (cron)	At clock times you set (e.g. daily 6am, hourly)	Time-based
File arrival	New files land in a UC external location or volume	Data-driven (event)
Table update	One or more source Delta tables get updated	Data-driven (event)
Continuous	Keeps the job running, restarting as it finishes	Always-on
Model update	A UC-registered model is created/updated	Data-driven (event)
Manual / API	"Run now", REST API, or another job	On-demand

Scheduled (time-based)

```
# Quartz cron: run every day at 06:00
0 0 6 * * ?
# seconds minutes hours day-of-month month day-of-week
```

Set a schedule with a cron expression and a timezone. Good when data is reliably ready by a known time.

File arrival trigger

Databricks polls a Unity Catalog external location or volume; when new files appear, the job runs. Ideal for data that lands on an irregular schedule. Enabling file events on the external location makes detection more efficient (and also speeds up Auto Loader).

Easy picture: a file-arrival trigger is a doormat sensor. Instead of checking the porch every hour on a timer, the job wakes the instant a package is dropped — no wasted checks, no waiting until the next scheduled sweep.

Table update trigger

The job runs as soon as a specified source table is updated — so downstream work starts when new data is genuinely ready, without a continuously running cluster or a guessed schedule.

Exam tip: file arrival = react to files landing in a volume/external location. Table update = react to a Delta table changing. Both are event-driven and avoid polling waste; continuous = a job that always runs.

Subdomain 4.4 — Choose between time-based and data-driven triggers

> *Choose between time-based and data-driven triggers based on data availability and pipeline dependencies.*

The core decision: a time-based (scheduled) trigger fires on the clock whether or not new data exists. A data-driven trigger (file arrival / table update) fires only when the data it depends on actually appears or changes.

Situation	Choose	Why
Source lands reliably at a fixed time each day	Scheduled (cron)	Predictable; simplest
Files arrive at unpredictable times	File arrival	Runs exactly when data shows up; no empty runs
Downstream must run right after an upstream table refreshes	Table update	Chains on real data readiness, not a guessed lag
Near-real-time processing needed	Continuous / streaming	Always processing new data
Ad-hoc / backfill	Manual / API	Run on demand

Easy picture: scheduled is checking the mailbox at 5pm every day — sometimes it's empty (wasted trip), sometimes mail came at noon and waited hours. Data-driven is the mail carrier ringing your bell the moment they deliver: no empty trips, no delay.

Exam tip: the two failure modes of pure scheduling: (1) empty runs — the job fires but no new data arrived (wasted compute); (2) stale data — data arrived right after the run, so it waits until the next tick. Data-driven triggers eliminate both. Choose time-based only when arrival timing is genuinely predictable.

The contrast people get wrong — continuous vs data-driven: continuous keeps a cluster always running (highest cost, lowest latency). A table-update or file-arrival trigger spins compute up only when there's work, then stops — much cheaper for bursty/irregular data while still being responsive. Reserve continuous for true near-real-time SLAs.

Deeper Dive: Lakeflow Jobs Internals

The basics above get you a working DAG; these details decide the tougher orchestration questions — run-if rules, passing data between tasks, repair runs, and the full trigger picture, with extra examples.

Run-if conditions (the fan-in rule you must know)

Run-if	Task runs when…
All succeeded (default)	Every dependency succeeded
At least one succeeded	≥1 dependency succeeded (others may fail)
None failed	No dependency failed (skipped is OK)
All done	All dependencies finished, regardless of result
At least one failed / all failed	Used for cleanup / alerting branches

> **Exam tip:** "run a cleanup/notification task even if an upstream task failed" → set run-if to All done (or At least one failed for a failure-only branch). Default "All succeeded" would skip it when an upstream fails.

Passing data between tasks — task values & parameters

```
# Task A: publish a value
row_count = df.count()
dbutils.jobs.taskValues.set(key="rows", value=row_count)

# Task B: read the upstream value
rows = dbutils.jobs.taskValues.get(taskKey="taskA", key="rows", default=0)

# If/else condition task expression referencing the value:
{{tasks.taskA.values.rows}} > 0
```

Mechanism	Use
Task values	Pass small results between tasks in a run
Job parameters	Values set on the whole job (e.g. `env=prod`), referenced as `{{job.parameters.env}}`
Widgets	Notebook-level parameters (`dbutils.widgets.get`) a job task can populate

Repair run, notifications, concurrency

Feature	Why it matters
Repair run	Re-run only the failed tasks of a run (not the whole job) — saves time & cost after a partial failure
Notifications	Email/webhook/Slack on start, success, failure, or duration threshold
Max concurrent runs	Limits overlapping runs of the same job (e.g. prevent a slow run colliding with the next schedule)
Shared job cluster	One job cluster reused across tasks in a run — avoids repeated startup cost

> **Exam tip:** after a 6-task job fails on task 5, don't re-run all 6 — use Repair run to re-execute only the failed (and downstream) tasks. This is the cost-smart answer.

Worked Examples & Field Notes

Walkthrough: reading a task graph like the exam does

```
ingest ──▶ clean ──▶ build_gold ──▶ refresh_dashboard
                │
                └──▶ quality_check ──┘        # runs in PARALLEL with build_gold
```

Two things the exam tests from a picture like this. First, `build_gold` and `quality_check` share one parent (`clean`) and don't depend on each other, so they run at the same time. Second, `refresh_dashboard` has two parents, so it waits for both to finish (default run-if = "all succeeded"). Fan-out means parallel; fan-in means wait.

> **Easy picture:** a job DAG is a relay race with some runners going in parallel lanes. The final runner (the dashboard) can't start until every baton from every lane is in hand. One dropped baton upstream and the whole finish line is left waiting.

Walkthrough: retries fix flakiness, not bugs

```
Task: call_partner_api     |   retries: 3    |    retry interval: 2 min
```

If the partner API throws a 503 because it hiccuped, retry #1 or #2 usually sails through — that's a transient failure, exactly what retries are for. But if your SQL says `SELECT custmer_id` (typo), it will fail identically all three times, plus the original, and you'll have simply waited longer to see the same error.

> **Coffee break.** *Setting 10 retries on a task with a hard-coded typo is optimism as a deployment strategy. The bug does not get tired. It will out-stubborn your retry count every single time.*

Walkthrough: the trigger decision, out loud

The situation	Trigger	The reasoning
Source is reliably ready by 5am daily	Scheduled (cron)	Timing is predictable; simplest thing that works
Files arrive at random hours	File arrival	Fires exactly when data lands — no empty runs
Run right after an upstream table refreshes	Table update	Chains on real data readiness, not a guessed lag
Sub-minute latency SLA	Continuous	Always running — highest cost, lowest latency

> **The trap the exam sets:** a scheduled job has two failure modes — empty runs (it fires, no new data, you paid for nothing) and stale data (data arrived right after the run, waits hours for the next tick). Data-driven triggers kill both. Pick "scheduled" only when arrival timing is genuinely predictable.

> **Real talk.** *A cron job that runs every 5 minutes on data that arrives twice a day spends most of its life waking up, looking around, finding nothing, and going back to sleep — like a very expensive, very anxious guard dog.*

Walkthrough: repair run saves the day (and the budget)

A 6-task pipeline fails on task 5 at 3am. You fix the bug at 9am. Do you re-run all six? No — the first four succeeded and re-running them wastes time and money (and might re-process data). Use Repair run: it re-executes only the failed task and its downstream dependents.

> **Exam tip:** "cheapest way to finish a partially-failed run after a fix" is always Repair run, never "re-run the whole job." And to pass a value from one task to another, use task values (`dbutils.jobs.taskValues.set/get`) — tasks run in separate processes, so shared variables don't exist.

One-Minute Recap

Lakeflow Jobs = native orchestrator. Job = whole workflow, task = one step, dependencies = a DAG. Task types: notebook, SQL, dashboard, pipeline, Python/JAR/dbt, Run Job. Tasks with a shared parent and no link run in parallel; a task with two parents waits for both. Control flow: retries (transient failures only), If/else condition tasks (branch on a task value), for-each (loop over inputs), run-if rules. Prefer branching/looping in the DAG over hidden in-notebook code. Triggers: scheduled/cron (time-based), file arrival + table update + model update (data-driven/event), continuous (always-on), manual/API. Time vs data-driven: scheduled = fixed clock, risks empty runs or stale data; data-driven fires only when files land / tables change (no waste, responsive); continuous = lowest latency, highest cost. Choose time-based only when arrival timing is predictable. Compute: run production tasks on job clusters/serverless, not always-on all-purpose. Pass data between tasks with task values. Repair run = re-run only failed tasks.

DOMAIN 5

Implementing CI/CD

10% of the exam · roughly 5 questions

Real data teams do not click buttons in production. They develop in Git, package their work, and promote the same code across environments automatically. This chapter covers the three pillars the exam tests: Git Folders for version control inside the workspace, Automation Bundles for packaging and promoting assets, and the Databricks CLI that drives it all in a CI pipeline.

Start Here — CI/CD From Absolute Zero (read this first)

If the words "bundle," "branch," or "promote" already feel like jargon, this section is for you. We'll build every idea from scratch, in plain language, before touching any Databricks-specific tooling. Nothing below assumes you've done this before.

What problem is CI/CD even solving?

Imagine you write some code that cleans data. It works on your screen. Now you need to get that exact code running for real, on a schedule, without breaking anything that's already working — and you'll need to do this again next week, and the week after. Doing that by hand (copy-pasting notebooks, clicking around, hoping you didn't miss a step) is slow and error-prone. CI/CD is just a disciplined, mostly-automatic way to move code changes from your laptop into production safely and repeatably.

The letters stand for Continuous Integration (regularly combining everyone's code changes and checking they still work together) and Continuous Delivery/Deployment (automatically shipping those checked changes out to where they run). You do not need to memorize the phrase. You need the idea: change code → check it → ship it, the same way every time.

> **Easy picture:** think of publishing a book. You write and edit drafts privately, a proofreader checks them, and only the approved version goes to the printer. CI/CD is that pipeline for code: draft safely, check automatically, and only the approved version reaches "the printer" (production).

Building block 1: Git (saving and sharing code, safely)

Git is a tool that remembers every version of your code and lets a team work on it without overwriting each other. If you've ever used "Version history" in Google Docs plus "Track changes," you already understand 80% of Git. Here are the only words you need:

Term	In plain English
Repository (repo)	The project folder Git is tracking — all your code and its full history
Commit	A saved snapshot with a message, like "cleaned null emails." You commit whenever you finish a small chunk of work
Branch	A private copy of the code where you can experiment without affecting the main version. `main` is the trusted, live branch
Merge	Folding your branch's changes back into main once they're approved

Term	In plain English
Pull request (PR)	A formal "please review and merge my branch" request — where a teammate checks your work before it joins main
Push / Pull	Push = upload your commits to the shared server (GitHub, etc.); Pull = download the latest

> **Easy picture:** `main` is the master recipe everyone cooks from. A branch is you photocopying the recipe to try adding garlic without ruining dinner for the whole restaurant. A pull request is showing the head chef your garlic version; if they approve, you merge it into the master recipe and now everyone gets garlic.

> **Coffee break.** *Git has a reputation for being scary, but 90% of daily use is four moves: make a branch, commit your work, push it, open a pull request. The other 10% is Googling how to undo something at 5pm on a Friday. Everyone does it. Even the head chef.*

Building block 2: environments (dev, test, prod)

You don't test new code on real customers. So teams keep separate copies of their setup, called environments:

Environment	What it's for
Dev (development)	Your sandbox. Break things freely. Fake or sample data
Test (staging)	A rehearsal that mimics production, to catch problems before real users see them
Prod (production)	The real thing — real data, real dashboards, real consequences

Promoting code means moving the same code from dev → test → prod as it earns trust. The golden rule of this whole chapter: the code doesn't change between environments — only its settings do (which catalog, which warehouse, which schedule).

> **Easy picture:** dev/test/prod are three theaters. Dev is the rehearsal room where you mess up lines with no audience. Test is the dress rehearsal. Prod is opening night. Same script all the way through — you just change the venue, not the play.

Building block 3: how Databricks does these three things

Building block	Databricks tool	What it does
Git (version your code)	Git Folders (formerly Repos)	Clone a repo into the workspace; branch, commit, push, open PRs from the UI
Package & promote across environments	Automation Bundles (formerly Asset Bundles / DABs)	One config file describes your jobs/pipelines and their per-environment settings
Automate the whole thing	Databricks CLI	Commands a CI system runs to validate, deploy, and run bundles automatically

> **Exam tip:** that's the entire mental model — Git Folders = version code, Automation Bundles = package + promote, CLI = automate. If a question is about "branches / commits / PRs," it's Git Folders. If it's about "deploy the same code to dev/test/prod," it's Bundles. If it's about "a command in a CI pipeline," it's the CLI.

A day in the life: the everyday loop

```
1. You create a branch off main:          feature/clean-emails
2. You edit a notebook in the Git Folder and commit:      "lowercase + trim emails"
3. You push your branch and open a pull request
4. A teammate reviews it; an automated check (bundle validate) runs
5. Approved -> you merge into main
6. Merging into main kicks off deployment: the code is bundled and
   promoted to test, and after a final check, to prod
```

Notice you never manually copied anything into production. You made a small, reviewed change, and the pipeline carried it the rest of the way. That safety and repeatability is CI/CD. Everything in the rest of this chapter is just the detail of steps 4–6.

Real talk. *The entire reason this machinery exists is a very human problem: people forget steps, and Fridays are dangerous. Automating "check it, then ship it" means the boring parts happen the same way every time, so you can save your brain for the actual work.*

The Big Picture — Dev → Test → Prod

CI/CD is about moving one codebase reliably across environments. The exam's model: you develop in a workspace using Git folders (Repos), package everything as an Automation Bundle (formerly Databricks Asset Bundles / DABs), and promote the same bundle to dev, test, and prod targets using the Databricks CLI (usually driven by a CI system like GitHub Actions).

Easy picture: think of a stage play. The script (your code in Git) never changes between the rehearsal room, the preview night, and opening night. What changes is the venue and props (the target's catalog, warehouse, schedule) — supplied by bundle variables and overrides. Same script, different stage.

Exam tip: the golden rule of this domain — promote the same code, change only the configuration. Never fork the notebook per environment; parameterize it.

Subdomain 5.1 — Manage your code development workflow in the workspace UI

> *Manage your code development workflow within the Databricks workspace UI, including creating and switching between branches in Databricks Git Folders (formerly Databricks Repos), committing and pushing changes, and creating pull requests using Databricks Git integration.*

In one sentence: a Git Folder lets you do all the normal Git things (branch, commit, push, open a pull request) from inside the Databricks workspace, so your notebooks live in version control instead of only in the UI. You connect it once to a remote repo (on GitHub, GitLab, Azure DevOps, or Bitbucket), and from then on the workspace is just another place you edit that repo.

Easy picture: without a Git Folder, your notebooks are like documents saved only on one laptop — no history, no backup, no teamwork. A Git Folder is like putting that folder in a shared, versioned drive: every change is tracked, and teammates can work on their own copies (branches) without overwriting yours.

Action	In Databricks UI
Clone a repo	Add a Git folder pointing at the remote URL
Create / switch branch	Branch dropdown in the Git dialog
Commit & push	Stage changed files, write a message, commit + push
Pull	Pull latest from the remote branch
Open a pull request	Create the PR (opens/links to the Git provider)
Resolve conflicts	Handle in the provider or by re-pulling

The standard feature-branch flow

```
1. Create a feature branch off main          (feature/add-silver-clean)
2. Edit notebooks / code in the Git folder
3. Commit & push to the remote
4. Open a pull request into main
5. Review + CI checks pass -> merge
6. CI/CD deploys the merged code to the next environment
```

Exam tip: a Git folder authenticates to the provider via a personal access token / Git credential stored per user (or a service principal for automation). Work on feature branches, never commit straight to `main`; merges to main are what trigger deployment.

Missed-question focus — the correct order: after editing, the sequence is select the changed files → write a commit message → Commit & Push, and only then open a Pull Request. A PR is a request to merge commits that are already pushed, so you can't create it before committing. Order: commit & push, then PR.

The contrast people get wrong — Git shares code, not data: Repos version your notebooks and code. They do not move or share table data (that's Delta Sharing). Keep them straight.

Subdomain 5.2 — Environment config with Automation Bundle variables and overrides

> *Understand environment-specific configuration using Automation Bundle (formerly Databricks Asset Bundles) variables and overrides while promoting the same codebase across dev, test, and prod targets.*

First, what a bundle actually is (no jargon)

Forget the word "bundle" for a second. Picture the manual way of doing things: to set up a data pipeline you'd open the Databricks UI and click to create a job, add tasks, pick a cluster, set a schedule. Now you need the exact same thing in your test workspace, and again in production. So you click through it two more times, trying to remember every setting. Next month someone changes the schedule in prod but forgets test, and now your three environments quietly disagree. That drift is the disease. An Automation Bundle is the cure: instead of clicking, you write down what you want in one text file, and a single command builds it for you in whichever environment you point at.

That text file is called `databricks.yml`. It is a plain description — "I want a job named X, that runs notebook Y, on schedule Z" — plus a small section saying "and here's what's different in dev vs test vs prod." Nothing runs when you write the file; it's just the plan. You apply the plan later with a command.

Easy picture — the house blueprint. A bundle is an architect's blueprint. The blueprint describes one house (your job/pipeline). You can build that same house on three different plots of land — dev, test, and prod — from the same blueprint. The only things that change per plot are details like the street address and the paint color: those are the variables. You never redraw the blueprint for each plot; you just fill in the plot-specific blanks. "Deploying" is handing the blueprint to the builder and saying "build it on this plot."

Another way to see it — a save file. If you've played a video game, a bundle is a save file for your Databricks setup. The save file captures your whole configuration; loading it into any workspace recreates that setup exactly. Lose the workspace? Load the save file somewhere else and you're back.

The databricks.yml, read like a story

```
bundle:
  name: sales_etl

variables:
  catalog:
    description: Target catalog
    default: dev_catalog
  warehouse_id:
    default: abc123

resources:
  jobs:
    sales_job:
      name: sales_etl_${var.catalog}
      tasks:
          - task_key: clean
            notebook_task:
               notebook_path: ./src/clean.py

targets:
  dev:
    mode: development           # prefixes names, pauses schedules
    default: true
    variables:
      catalog: dev_catalog
  test:
    variables:
      catalog: test_catalog
  prod:
    mode: production
    variables:
      catalog: prod_catalog
      warehouse_id: prod_wh_999
    run_as:
      service_principal_name: etl-sp       # prod runs as a service principal
```

Notice what changed and what didn't. The job definition is written once. The only per-environment differences are the filled-in blanks (which catalog) and a couple of safety settings (dev pauses schedules; prod runs as a service principal). That is the entire point: one description, many environments, differences pushed into a tiny variables section.

Concept	Role
Resources	The assets to deploy: jobs, pipelines, schemas, dashboards…

Concept	Role
Targets	Named environments (dev/test/prod) with their own workspace + overrides
Variables	Placeholders (`${var.x}`) swapped per target — catalog, warehouse, paths
Overrides	Target-specific values that replace defaults
`mode: development`	Prefixes resource names with your username, pauses schedules/triggers — safe iterating
`mode: production`	Deploys "for real"; commonly runs as a service principal

Easy picture: variables are the fill-in-the-blanks on a form letter. "Dear ___, welcome to ___." One template, and each target fills the blanks with its own catalog, warehouse, and schedule.

The contrast people get wrong — dev mode vs prod mode: development mode isolates your deploys (username-prefixed names, paused schedules) so parallel devs don't collide and nothing fires by accident. production mode uses clean names and typically a service principal. Rule: iterate in dev mode, deploy the real thing in prod mode.

The contrast people get wrong — target overrides vs a cluster policy: to give each environment different resource settings (e.g. a bigger `node_type_id` in prod), you override them inside the targets block of the same `databricks.yml` — the bundle keeps one codebase and swaps properties per target. A global cluster policy is a workspace-level admin guardrail, not the bundle mechanism for per-environment config. Rule: "different node type / cluster size / setting per dev-test-prod, one codebase" → targets block overrides, not a cluster policy.

Subdomain 5.3 — Deploy Declarative Automation Bundles

> *Deploy Declarative Automation Bundles (formerly Databricks Asset Bundles) to package, configure, and promote Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and other workspace assets across dev, test, and prod environments.*

What "deploy" means, concretely: deploying reads your `databricks.yml` and actually creates or updates those jobs and pipelines in a real workspace. Before deploy, the file is just a plan on your disk. After `databricks bundle deploy -t test`, there is a real job sitting in the test workspace, configured exactly as the file described. Run it again after editing the file and it updates the existing job rather than making a duplicate — the file is the single source of truth.

A bundle packages, configures, and promotes jobs, Lakeflow Declarative Pipelines, and other workspace assets across environments. The lifecycle is validate → deploy → run.

```
# validate the bundle config (syntax + resolves variables)
databricks bundle validate -t dev

# deploy resources to a target environment
databricks bundle deploy -t test

# run a job/pipeline defined in the bundle
databricks bundle run sales_job -t test

# promote to prod (same bundle, prod target)
databricks bundle deploy -t prod
```

Exam tip: the same bundle is deployed to each target with just `-t <target>`. That single flag swaps catalog, warehouse, run-as identity, and schedules, proving "one codebase, many environments". You do not maintain separate copies per env.

Easy picture: writing the bundle file is drawing the blueprint; deploy is the construction crew building it on the plot you named with `-t`. `run` is then flipping the light switch to make sure it works.

Subdomain 5.4 — The Databricks CLI to validate, deploy, and manage bundles

> *Understand the Databricks CLI to validate, deploy, and manage Declarative Automation Bundles (formerly Databricks Asset Bundles) and other workspace assets in automated CI/CD workflows.*

What the CLI is, plainly: the Databricks CLI (command-line interface) is just the set of typed commands — like `databricks bundle deploy` — that do things to your workspace without clicking. You (or, more importantly, an automated system) run these commands to validate, deploy, and run bundles. The reason it matters for CI/CD is that a robot can't click buttons, but it can run commands. So when someone merges code, an automated pipeline (GitHub Actions, Azure DevOps) runs these CLI commands for you, as a service principal, and your change flows to production hands-free.

Command	Does
`databricks bundle validate`	Checks config is valid before deploying (use in CI as a gate)
`databricks bundle deploy`	Creates/updates the target's resources
`databricks bundle run`	Triggers a job/pipeline run (e.g. integration tests)
`databricks bundle destroy`	Tears down deployed resources
`databricks auth login`	Configures auth (OAuth / PAT / service principal)

```
# Typical CI job on merge to main (pseudo GitHub Actions):
- databricks bundle validate -t prod        # gate: fail fast on bad config
- databricks bundle deploy     -t prod       # promote
- databricks bundle run tests -t prod        # smoke/integration run
```

The contrast people get wrong — CLI/bundles vs clicking in the UI: configuring jobs by hand in each workspace UI is manual, drifts between environments, and isn't reproducible. Bundles + CLI make deployments declarative, versioned, and automated — the same command produces the same result every time. Rule: "automate promotion across dev/test/prod" → bundles deployed via the CLI, never manual UI setup per env.

Exam tip: CI/CD automation should authenticate as a service principal, not a person — so pipelines keep working when someone leaves and permissions are scoped to the robot identity. This mirrors the "run production jobs as a service principal" rule.

Weak-Area Clinic — Exact Commands, Flags & Keywords (memorize these)

The CI/CD questions that trip people up are not conceptual — they test whether you know the exact command name, the exact flag, and the exact keyword. Wrong-but-plausible names (`bundle show`, `bundle describe`, `-v`, `dict`) are the classic distractors. This clinic drills the real ones.

1 · *The three read-only bundle commands (validate vs plan vs summary)*

Command	Answers the question...	What you get
`databricks bundle validate`	"Is my config sound?"	Syntax/schema check, resolves variables; fails fast on errors
`databricks bundle plan`	"What would change?"	A preview of the create/update/delete actions — without deploying
`databricks bundle summary`	"What did it resolve to?"	A human-readable overview of the bundle's identity and resources (with deep links)

Trap — the fake commands: there is no `bundle show` and no `bundle describe`. These are the seductive wrong answers. To preview changes it's `plan`; for a human-readable overview it's `summary`. If an option's command name isn't one of validate / plan / summary / deploy / run / destroy, be very suspicious.

Missed-question focus — what bundle destroy deletes: `databricks bundle destroy` removes the resources the bundle deployed to the workspace (the jobs, pipelines, etc.). It does not touch your local files — your `databricks.yml` and code stay on disk. All bundle commands act on workspace resources, not your local project.

Exam tip — the trigger words: "preview the specific changes (creations, updates, deletes) without deploying" → `bundle plan`. "human-readable overview of the bundle's identity and resources" → `bundle summary`. "check the config is valid before deploying" → `bundle validate`.

2 · *Injecting a variable at deploy time (the flag is --var, with =)*

```
# RIGHT — override the 'catalog' variable at deploy time
databricks bundle deploy -t test --var="catalog=test_catalog"

# override several at once
databricks bundle deploy -t test --var="catalog=test_catalog,warehouse_id=abc123"

# WRONG — these are the distractors, none of them work:
#    -v catalog:test_catalog         (-v is usually verbose/version, and ':' is wrong)
#    --var catalog:test_catalog      (colon separator is wrong — must be '=')
```

Trap — -v vs --var: `-v` is not variable injection — across most CLIs it means verbose or version. The variable flag is the spelled-out `--var`, and the separator between name and value is an equals sign (`name=value`), never a colon. Remember: --var, equals, in quotes.

3 · *Structured variable types — the keyword is complex (not dict)*

```
variables:
    cluster_tags:
```

```
      description: Tags applied to the job cluster
      type: complex                  # REQUIRED for an object/map/list value
      default:
        team: data-eng
        cost_center: "4470"

# then reference the whole object elsewhere:  ${var.cluster_tags}
```

Trap — complex, not dict: `dict` is a Python word; it is not valid in a bundle's YAML schema. When a variable holds a map/object/list (structured data), the type keyword is complex. Simple values (a string, a number) need no type at all. So: structured value → `type: complex`.

4 · *Git for beginners — commit/push/pull are safe; rebase is discouraged*

Operation	Beginner-safe?	Why
Commit	Yes	Saves a snapshot — additive, reversible
Push / Pull	Yes	Uploads/downloads commits — no history rewriting
Create / switch branch, merge	Yes	Standard, safe collaboration
Rebase	No — discouraged	Rewrites commit history, which can confuse teammates and lose work if done wrong

Trap: the "NOT recommended for beginners" answer is rebase, because it rewrites history. Committing, pushing, and pulling are all perfectly safe — don't get talked into picking one of those.

Coffee break. *Rebase is the power tool of Git: incredible in skilled hands, and a great way to remove a finger if you've never used one. Beginners: commit, push, pull, merge. Leave the rebase chainsaw in the shed for now.*

5 · *Where to read commit history — the Git dialog's History tab*

To see who made the last commit and read its message inside Databricks, you open the Git dialog (the same panel where you branch, commit, and push) and click the History tab. That's the standard, and only, built-in path.

Trap: there is no right-click / context-menu "view history" option on the notebook. The commit history lives in the Git dialog → History tab. If an option describes a context-menu item for history, it's the invented distractor.

Exam tip: "see who made the most recent commit and read the message, in the workspace UI" → open the Git dialog, go to the History tab.

Deeper Dive: Bundle Anatomy, Git Folders & the CLI

Anatomy of an Automation Bundle

Key	Role
`bundle`	Bundle name/metadata

Key	Role
`variables`	Reusable placeholders (`${var.catalog}`) overridden per target
`resources`	The assets to deploy: jobs, pipelines, schemas, dashboards, etc.
`targets`	Named environments (dev/test/prod) with per-target overrides & workspace host
`include`	Pull in resource YAML from other files (keeps it modular)

Bundle substitutions, APIs/SDK & secrets (favourites)

Substitution in databricks.yml	Resolves to
`${bundle.target}`	The current deploy target name (e.g. `staging`) — great for per-env schema names
`${bundle.name}`	The bundle's name
`${workspace.current_user.userName}`	The deploying user's email (unique per-user paths in dev)

Programmatic access	How
Clone a repo (REST)	`POST /api/2.0/repos` with `url`, `provider`, `path`
Switch a Git folder's branch (REST)	`PATCH /api/2.0/repos/{id}` with a branch payload (syncs latest remote commit)
Python SDK jobs	`jobs.Task(task_key=…, depends_on=[jobs.TaskDependency(...)], run_if=ALL_SUCCESS)` — depends_on + run_if gate execution

> **Secrets:** store credentials in secret scopes, never in code. CLI: `databricks secrets list-scopes`, then `databricks secrets put-secret <scope> <key> --string-value "…"`. Read in code with `dbutils.secrets.get(scope, key)`. Crucial: list-secrets and the API return only key names / metadata — never the actual secret values.

Worked Examples & Field Notes

Walkthrough: one bundle, three environments

```
databricks bundle validate -t dev          # 1. check config, resolve variables (CI gate)
databricks bundle deploy   -t dev          # 2. deploy to dev: names prefixed, schedules paused
databricks bundle deploy   -t test         # 3. same code, test catalog/warehouse
databricks bundle deploy   -t prod         # 4. same code, prod catalog, runs as service principal
```

The only thing that changed across those four lines is the `-t` flag. That one flag swaps the catalog, the warehouse, the run-as identity, and whether schedules are live. That is the entire philosophy of the chapter in one command.

> **Coffee break.** *"It works on my machine" is not a deployment strategy, it's a confession. Bundles exist so that the thing that ran in dev is byte-for-byte the thing that runs in prod, and the only variable left to blame is you.*

Walkthrough: why dev mode saves you from yourself

In a bundle target, `mode: development` does two quietly heroic things. It prefixes every resource name with your username, so when five engineers deploy the same job to the same workspace, they don't stomp on each other — you get `chris_sales_job`, not a fistfight over `sales_job`. And it pauses all schedules and triggers, so your half-finished experiment doesn't fire at 3am and email the whole company.

Walkthrough: the Git workflow that triggers deployment

```
1. git checkout -b feature/add-silver-clean     # branch off main
2. edit notebooks in the Git Folder, commit, push
3. open a pull request into main
4. review + CI (bundle validate) passes -> merge
5. merge to main triggers the deploy pipeline
```

> **Real talk.** *Committing straight to main on a Friday afternoon is how legends are made, and by "legends" I mean the incident report with your name in it. Branch. Please.*

One-Minute Recap

Golden rule: promote the same code, change only config. Git folders (Repos): clone a remote repo into the workspace; create/switch branches, commit, push, open PRs from the UI; work on feature branches, merge to main triggers deploy; Git shares code not data. Automation Bundles (formerly Databricks Asset Bundles): `databricks.yml` declares resources (jobs, pipelines…), targets (dev/test/prod), variables + overrides. `mode: development` = username-prefixed names + paused schedules; `mode: production` = clean names, runs as a service principal. Lifecycle: `bundle validate` → `deploy -t <target>` → `run`; the `-t` flag swaps catalog/warehouse/run-as. Read-only trio: validate ("is config sound?"), plan ("what would change?"), summary ("what did it resolve to?") — no `show` / `describe`. Override a variable with `--var="key=value"` (not `-v`, not `:`); structured variable → `type: complex` (not `dict`). `bundle destroy` removes workspace resources, not local files. Rebase is discouraged for beginners; commit history = Git dialog History tab. Databricks CLI drives it in CI (GitHub Actions/Azure DevOps), authenticating as a service principal.

DOMAIN 6

Troubleshooting, Monitoring and Optimization

10% of the exam · roughly 5 questions

When a pipeline slows down or fails, you need to know where to look and what the symptoms mean. This chapter teaches you to read job run history for trends, interpret the Spark UI to spot skew, shuffle, and spill, diagnose the common cluster failures, and apply the modern optimization features that keep tables fast without constant manual tuning.

Two Layers of Monitoring — Job level vs Spark level

This domain splits cleanly. The Lakeflow Jobs UI tells you which task is slow or failing and how today compares to history (the orchestration view). The Spark UI tells you why a specific stage is slow — skew, shuffle, spill (the engine view). Then there are table optimizations (Liquid Clustering, predictive optimization) and cluster-level failures (startup, libraries, OOM).

> **Easy picture:** the Jobs UI is the delivery-tracking app — it shows which leg of the trip is late. The Spark UI is popping the hood on that one truck to see the engine problem. You use the tracker to find the slow leg, then the engine view to diagnose it.

Subdomain 6.1 — Identify trends in job performance (Run History)

> *Identify trends in job performance using the Lakeflow Jobs run history view to compare current execution times against historical baselines.*

The Lakeflow Jobs run history lists every past run with start time, duration, and status. You compare current runtime against historical baselines to spot a job that's slowly (or suddenly) degrading.

Symptom in run history	Likely meaning
Gradual creep in duration over weeks	Growing data volume; time to tune/optimize/cluster
Sudden jump on a specific date	A code change, data skew event, or config change
High variance run-to-run	Contention, spot reclamation, or skew that depends on data
Rising failure rate	Upstream data issues, OOM, or flaky dependency

> **Exam tip:** the run history view is for trend/baseline comparison — "is this run slower than usual?" It is the first place you look before diving into Spark internals. A baseline is just the typical historical duration.

Subdomain 6.2 — Use the Lakeflow Jobs UI to monitor pipeline health

> *Use the Lakeflow Jobs UI to monitor pipeline health by interpreting job statuses, viewing DAG-based task graphs to spot upstream blockers, and tracking pipeline run times and failure rates.*

Signal	How to read it
Task status colors	Success / running / failed / skipped — spot the failed task instantly
DAG task graph	See dependencies; a failed upstream task is the blocker for everything downstream
Run duration & timeline	Which task dominates the runtime (the bottleneck task)
Failure rate	How often the job fails over time

Easy picture: the DAG graph is a subway map during an outage. One red station upstream stops every train down the line. You don't fix the empty downstream platforms — you fix the blocked station causing it.

The contrast people get wrong — root cause vs symptom: when many downstream tasks fail or are skipped, the culprit is usually a single failed upstream task. Read the DAG top-down to find the first failure — that's the root cause. Fixing downstream tasks that were merely blocked wastes time.

Exam tip: "several tasks failed/were skipped — where do you look?" → the DAG graph for the earliest upstream failure. "Is the job trending slower?" → run history.

Subdomain 6.3 — Identify common performance bottlenecks

> *Identify common performance bottlenecks such as data skew, shuffling, and disk spilling by interpreting stage-level metrics in the Spark UI.*

The Spark UI shows stage-level metrics. Three classic bottlenecks, and how each looks:

Bottle-neck	Symptom in Spark UI	Fix
Data skew	One task runs far longer than the rest in a stage (max task time ⩾ median); uneven shuffle read sizes	AQE skew join handling, salting, broadcast the small side, Liquid Clustering
Shuffle	Large "Shuffle Read/Write" bytes; long exchange stages	Reduce data before shuffle, broadcast join, fewer wide ops, tune shuffle.partitions
Disk spill	Non-zero "Spill (memory)" / "Spill (disk)" in stage metrics	More executor memory, more/right-sized partitions, less data per task

Easy picture: imagine sorting mail into bins. Skew is one bin getting 90% of the letters while the rest sit idle — one worker drowns. Shuffle is carrying letters across the whole room to reach the right bin — expensive movement. Spill is your desk overflowing so you dump piles on the floor (disk) and keep walking back for them — slow.

The contrast people get wrong — skew vs plain slowness: if every task in a stage is slow, you have a sizing/volume problem. If one task takes 10× the others while the rest finished long ago, that's skew — one partition has way too much data. Look at the task-duration distribution (min/median/max), not just the total.

Exam tip: spill = you ran out of memory and wrote to disk (slow but not a crash). OOM = you ran out of memory and the executor died. Spill is a performance warning; OOM is a failure. Adaptive Query Execution (AQE) auto-handles many skew/partition problems when enabled.

Subdomain 6.4 — Liquid Clustering & predictive optimization

> *Understand the features of Liquid Clustering and predictive optimization.*

Liquid Clustering

Liquid Clustering (`CLUSTER BY`) is the modern replacement for both Hive-style partitioning and ZORDER. It physically organizes data by chosen keys so queries filtering on those keys skip irrelevant files (data skipping).

Feature	Liquid Clustering	Old partitioning / ZORDER
Change keys later	Yes — without rewriting the whole table	Partitioning: no (must rewrite)
Small-file / skew problems	Handles them automatically	Over-partitioning causes tiny files
Setup	`CLUSTER BY (col)`	Choose partition cols + run ZORDER

```
CREATE TABLE main.silver.events (...) CLUSTER BY (event_date, user_id);
-- or let Databricks pick keys automatically:
CREATE TABLE main.silver.events (...) CLUSTER BY AUTO;
-- compact + cluster existing data:
OPTIMIZE main.silver.events;
```

Easy picture: old partitioning is nailing rigid dividers into a filing cabinet — pick the wrong split and you must rip it all out and refile. Liquid Clustering is smart folders that reorganize themselves as data grows, and you can change the sort key without redoing the cabinet.

Predictive Optimization

Predictive optimization lets Databricks automatically run maintenance — `OPTIMIZE` (compaction/clustering) and `VACUUM` (removing old files) — on Unity Catalog managed tables, when its model decides it's worthwhile. You stop scheduling maintenance jobs by hand.

Missed-question focus — what compute runs it: predictive optimization's background maintenance runs on Databricks-managed serverless compute, not your all-purpose or job clusters. You don't provision or pay to keep a cluster up for it — Databricks handles the compute. Exam tell: "what compute does predictive optimization use?" → serverless (managed by Databricks).

The contrast people get wrong — manual vs predictive maintenance: traditionally you schedule nightly `OPTIMIZE` / `VACUUM` jobs and guess the cadence. Predictive optimization observes access patterns and runs maintenance only when beneficial, cutting both cost and stale-layout problems. Rule: "automatically keep tables optimized without manual jobs" → predictive optimization.

Exam tip: at Associate level, recognize the value props: Liquid Clustering = flexible, changeable clustering replacing partitioning+ZORDER; predictive optimization = automatic OPTIMIZE/VACUUM on UC managed tables. Both reduce manual tuning.

Subdomain 6.5 — Diagnose cluster startup, library & OOM failures

> *Diagnose cluster startup failures, library conflicts, and out-of-memory issues.*

Failure	Common causes	Fix direction
Cluster startup failure	Cloud capacity/quota limits, bad instance type, permission/network (VPC, IAM) issues, invalid init script	Check event log; raise cloud quota; fix instance/policy; fix init script
Library conflict	Two libraries need incompatible versions; wrong package for the runtime	Pin compatible versions; use a matching DBR; isolate deps
Out of memory (OOM)	Executor: too much data per partition, skew, huge shuffle; Driver: `collect()` of big data, oversized broadcast	More memory, more partitions, avoid `collect`, fix skew

Easy picture: a driver OOM is trying to pour the whole swimming pool into a coffee cup — `collect()`-ing a massive result to the single driver node overflows it. Keep big data distributed across executors; only bring back small summaries.

Missed-question focus — mapPartitions iterator vs list: `mapPartitions` hands your function a lazy iterator over the partition's rows. If your code does `list(iterator)` (or otherwise materializes it all at once), you pull the entire partition into memory → executor OOM. Fix by keeping it lazy (iterate / `yield` row by row), not by tuning shuffle.partitions — this is a memory bug, not a shuffle problem.

The contrast people get wrong — driver OOM vs executor OOM: driver OOM comes from pulling data to the driver: `collect()`, `toPandas()`, or a broadcast that's too big. Executor OOM comes from too much data per task: skew or too few partitions. Rule: if it dies during `collect`/broadcast → driver; if it dies mid-shuffle/stage → executor.

Missed-question focus — init scripts & library conflicts: if the cluster event log shows `INIT_SCRIPTS_STARTED` → `INIT_SCRIPTS_FINISHED (FAILED)`, the exact error is in the init script logs in the configured cloud-storage path (the failing command + exit code). For pip installs in init scripts, use `pip install --no-cache-dir` to avoid stale/corrupt cached wheels causing library conflicts.

Exam tip: the cluster event log and driver logs are where startup and OOM causes surface. A library conflict typically shows an import/version error at attach or job start. Startup failures often trace to cloud-side limits (quota, capacity, IAM), not your code.

Deeper Dive: Reading the Spark UI & Fixing Skew

The official Section 6 sample question is a skew-diagnosis problem. This walks the exact stage-metric signatures, the three bottlenecks side by side, cluster-failure buckets, and optimization features — with extra practice.

Diagnosing skew from stage metrics (the official sample Q)

Scenario: a job doubled in duration; the longest stage shows most tasks finish in <30s but one task takes >10 min, and the task summary shows Min/Median shuffle read ~400 MB while Max shuffle read >5 GB. That signature — one giant task, huge Max vs Median — is data skew: one partition holds far too much data.

Option	Verdict
Confirm AQE with skew join handling is active (auto-splits the oversized partition)	Correct — targets the actual skew, no code change
Add more executors (bigger cluster)	

Option	Verdict
	Wrong — the one skewed task still runs alone; others sit idle
Lower shuffle.partitions (coalesce into fewer tasks)	Wrong — makes skew worse (more data per task)
Manually salt the skewed key before the join	Works, but AQE does it automatically — the intended answer is enabling AQE

Exam tip: the tell for skew is Max task time / shuffle read ⩾ Median. First-line fix = Adaptive Query Execution (AQE) with skew join handling (on by default in modern DBR; it splits the oversized partition at runtime). Salting is the manual fallback. Bigger clusters and fewer partitions do not fix skew.

Cluster failure diagnosis (memorize the buckets)

Failure	Likely cause	Where to look / fix
Startup failure	Cloud quota/capacity, bad instance type, IAM/network, bad init script	Cluster event log; raise quota; fix policy/init script
Library conflict	Incompatible versions / wrong package for the runtime	Pin compatible versions; match the DBR; import error at attach
Driver OOM	`collect()`, `toPandas()`, oversized broadcast	More driver memory, or don't collect big data
Executor OOM	Skew, too few partitions, huge shuffle	More executor memory / more partitions / fix skew

The contrast people get wrong — driver vs executor OOM: dies during `collect` / `toPandas` /broadcast → driver. Dies mid-shuffle/stage → executor. Startup failures usually trace to cloud-side quota/capacity/IAM, not your Spark code.

Optimization features

Feature	What it does
Liquid Clustering (`CLUSTER BY`)	Replaces partitioning + ZORDER; keys changeable without rewrite; auto-handles small files/skew
Predictive optimization	Auto-runs OPTIMIZE/VACUUM on UC managed tables when beneficial — no manual maintenance jobs
Photon	C++ vectorized engine; speeds SQL/DataFrame ops (not Python UDFs); higher $/hr but usually cheaper overall
AQE	Runtime: coalesces shuffle partitions, switches join strategy, handles skew

Exam tip: "keep tables optimized automatically without scheduling jobs" → predictive optimization. "Organize a table by filter columns, change keys later without rewriting" → Liquid Clustering. "Job trending slower over weeks" → Lakeflow Jobs run history vs baseline. "Which task blocks the rest" → the DAG graph, find the earliest upstream failure.

Worked Examples & Field Notes

Walkthrough: reading a skewed stage in the Spark UI

```
Task duration:     min 8s     median 12s   max 11 min        # one task » all others
Shuffle read:      min 380MB median 400MB max 5.1 GB         # one partition is enormous
```

That fingerprint — everyone finished, one straggler is still grinding, and its shuffle read dwarfs the median — is data skew. One key (say, `country = 'US'`) has vastly more rows than the rest, so its partition is a boulder while the others are pebbles.

Tempting "fix"	Does it help?
Add more executors (bigger cluster)	No — the one giant task still runs alone; new executors sit idle
Lower shuffle.partitions	No — fewer, bigger partitions makes skew worse
Enable AQE skew join handling	Yes — it splits the oversized partition at runtime
Manually salt the skewed key	Works, but it's the manual version of what AQE does automatically

Easy picture: skew is a checkout line where one shopper has 900 items and everyone else has three. Opening more registers (more executors) doesn't help the poor cashier stuck with the 900. You need to split that one giant cart across lanes — which is exactly what AQE skew handling does.

Coffee break. *Throwing a bigger cluster at data skew is like hiring more waiters for a restaurant where one table ordered everything on the menu and the kitchen has one chef. The bottleneck was never the number of waiters.*

Walkthrough: spill vs OOM (the calm one vs the fatal one)

```
Spill (disk): 4.2 GB           # job SLOWED — ran out of RAM, used disk, kept going
...
ExecutorLostFailure: OOM       # job DIED — ran out of RAM, no fallback, process killed
```

Spill vs OOM: spill is a performance warning — annoying, survivable, fix with more memory or more partitions. OOM is a funeral — the process is gone. Same root cause (not enough memory), very different severity.

Walkthrough: which node ran out of memory?

```
df.toPandas()        # dies HERE -> DRIVER OOM (pulling all data to one node)
df.collect()         # same — DRIVER
big_join.count()     # dies mid-stage -> EXECUTOR OOM (skew / too few partitions)
```

Exam tip: died during `collect` / `toPandas` /broadcast → driver memory. Died mid-shuffle/stage → executor memory. And if a cluster won't even start, that's usually cloud-side (quota, capacity, IAM), not your code — check the event log.

Real talk. `toPandas()` *on a 200 GB DataFrame is asking one laptop-sized driver to hold what a whole cluster was carrying. It's the "hold my drink" of Spark — briefly confident, immediately catastrophic.*

One-Minute Recap

Two layers: Jobs UI = which task is slow/failing; Spark UI = why a stage is slow. Run history compares current duration to a baseline (spot creeping/sudden regressions). Jobs UI health: task status colors + DAG graph — a failed upstream task is the blocker/root cause for downstream skips; read top-down. Spark UI bottlenecks: skew = one task ⋙ others (fix: AQE skew handling, salting, broadcast, clustering); shuffle = big shuffle read/write (fix: broadcast, filter early, tune `shuffle.partitions`); spill = memory overflow to disk (slow, not a crash). Liquid Clustering (`CLUSTER BY`) replaces partitioning + ZORDER; keys changeable without rewrite. Predictive optimization auto-runs OPTIMIZE/VACUUM on UC managed tables (serverless). Failures: startup = cloud quota/capacity/IAM/init script (check event log); library conflict = version mismatch (pin/align DBR); OOM — driver (`collect`/broadcast too big) vs executor (skew/too few partitions); `list(iterator)` in mapPartitions = executor OOM. Spill = warning, OOM = failure.

DOMAIN 7

Governance and Security

15% of the exam · roughly 7 questions

Governance decides who can see and do what — and Unity Catalog makes those rules consistent everywhere. This chapter covers the difference between managed and external tables, the precise mechanics of granting, revoking, and denying access, how to hide sensitive rows and columns, and how ABAC scales all of that across thousands of tables at once. It is conceptual, high-value, and very learnable.

The Governance Model — Unity Catalog Recap

All of Domain 7 lives inside Unity Catalog. Access is granted on a hierarchy, and privileges inherit downward: a grant on a catalog applies to all its schemas and tables unless overridden.

```
Metastore
 └─ Catalog            # grant here -> applies to everything below
       └─ Schema
          └─ Table / View / Volume / Function / Model
```

Easy picture: think of a building. The metastore is the building, catalogs are floors, schemas are rooms, tables are cabinets. A keycard that opens a whole floor (catalog grant) opens every room and cabinet on it — unless a specific room has its own stricter lock.

Exam tip: to query a table a user needs `USE CATALOG` on its catalog, `USE SCHEMA` on its schema, and `SELECT` on the table. Missing any link in the chain = access denied.

Subdomain 7.1 — Managed vs external tables in Unity Catalog

> *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

	Managed table	External table
Who controls storage	Unity Catalog (default managed location)	You — data sits at a path you specify (`LOCATION`)
DROP TABLE deletes data?	Yes — metadata and underlying files	No — only metadata; files remain at the path
Format	Delta (managed optimizations, predictive optimization)	Delta / Parquet / other
Best for	Default choice; UC handles layout & maintenance	Data shared with external tools, or an existing data lake path

```
-- managed: UC owns the storage
CREATE TABLE main.silver.orders (id INT, amount DOUBLE);

-- external: you point at a path (needs an external location + credential)
```

```
CREATE TABLE main.silver.ext_orders (id INT, amount DOUBLE)
  LOCATION 's3://my-bucket/orders/';

-- convert an existing external table to managed (and vice versa in recent UC)
ALTER TABLE main.silver.ext_orders SET MANAGED;
```

The contrast people get wrong — DROP behavior: dropping a managed table deletes the data files too. Dropping an external table leaves the files untouched — only the UC registration goes away. Rule: "data must survive dropping the table" or "path managed by another system" → external. Default, let-UC-manage-it → managed.

Exam tip: external tables require an external location (a registered path) backed by a storage credential. Managed tables just use the metastore/catalog's default storage. Volumes have the same managed/external split for files.

Subdomain 7.2 — Configure access controls using the UI and SQL

> *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

Statement	Effect
GRANT	Give a privilege to a principal
REVOKE	Remove a previously granted privilege (back to neutral)
DENY	Explicitly block — overrides any GRANT (hard "no")

Principals = users (people, by email), groups (collections of users — the recommended grant target), and service principals (robot identities for automation).

```
GRANT USE CATALOG ON CATALOG main TO `analysts`;
GRANT USE SCHEMA   ON SCHEMA main.sales TO `analysts`;
GRANT SELECT ON TABLE main.sales.orders TO `analysts`;

REVOKE SELECT ON TABLE main.sales.orders FROM `analysts`;
DENY    SELECT ON TABLE main.sales.salaries TO `contractors`;        -- hard block

SHOW GRANTS ON TABLE main.sales.orders;      -- audit current privileges
```

Easy picture: GRANT hands out a key. REVOKE takes the key back (you're just... keyless, neutral). DENY is a "DO NOT ENTER" sign bolted to the door that beats any key you might also hold. GRANT + DENY on the same object = DENY wins.

The contrast people get wrong — REVOKE vs DENY: REVOKE simply removes a grant (if another group grant still applies, the user may keep access). DENY is an explicit block that overrides grants, even inherited ones. Rule: "make sure this principal absolutely cannot access X even though they're in a group that can" → DENY. "Just take away this specific grant" → REVOKE.

Exam tip: best practice is to grant to groups, not individual users, and grant high in the hierarchy where inheritance makes sense. Precedence: an explicit DENY beats any GRANT. Also note `USE CATALOG` / `USE SCHEMA` are needed to traverse to an object.

The traversal rule (the #1 governance gotcha)

To query a table a principal needs all three links in the chain: `USE CATALOG` on the catalog, `USE SCHEMA` on the schema, and `SELECT` on the table. Miss any one → access denied.

> **Trap — SELECT granted but still denied:** the classic exam scenario is a user with SELECT who can't query because they lack `USE CATALOG` / `USE SCHEMA`. Crucial detail: `USE CATALOG` and `USE SCHEMA` are separate privileges — granting SELECT (even on the whole catalog) does not auto-include them. Privileges do inherit downward, so a common clean pattern is to grant `USE CATALOG` on the catalog plus `USE SCHEMA` and `SELECT` at the catalog level, and the latter two cascade to every schema and table. But `SELECT` alone is never enough on its own.

More privileges & governance objects the exam tests

Privilege / object	What it grants
READ FILES / WRITE FILES (on an External Location)	Direct file read/write on the cloud path (raw Parquet/JSON/CSV) — distinct from table SELECT/INSERT
READ VOLUME / WRITE VOLUME	Read/upload files in a UC volume (not SELECT/INSERT — those are table privileges)
MANAGE	Delegate administration: GRANT/REVOKE on the object without ownership or data access
ALL PRIVILEGES	Every applicable privilege on the securable

Lineage & system tables

Source	Answers
Column-level lineage	Records every source column that fed a derived column (a CTAS from in1+in2 records both)
`system.access.table_lineage` / `column_lineage`	Query read/write lineage events (filter `source_table_full_name`, `event_date`)
`system.billing.usage`	DBU consumption per compute/tag — the cost table
`system.access.audit`	Who accessed/changed what, when (investigations)
`system.lakeflow.job_run_timeline`	One row per job run (state, start/end, trigger) — for run-history trend analysis
`system.information_schema`	What exists now: tables, columns, grants (reports/inventory)

Subdomain 7.3 — Column-level masking and row-level security

> *Understand column-level masking and row-level security to restrict data visibility based on user groups.*

Sometimes different users should see the same table but different slices of it. Two mechanisms, both backed by UC functions:

Mechanism	Restricts	Example
Column mask	What a column shows (hide/obfuscate values)	Show ***-**-1234 instead of full SSN for non-HR

Mechanism	Restricts	Example
Row filter	Which rows a user sees	A regional manager sees only their region's rows

```
-- column mask: a UDF returns masked value unless user is in hr_team
CREATE FUNCTION mask_ssn(ssn STRING) RETURN
  CASE WHEN is_account_group_member('hr_team') THEN ssn
       ELSE 'XXX-XX-' || substr(ssn, -4) END;
ALTER TABLE employees ALTER COLUMN ssn SET MASK mask_ssn;

-- row filter: a UDF returns TRUE for rows the user may see
CREATE FUNCTION region_filter(region STRING) RETURN
  is_account_group_member('all_regions') OR region = current_user_region();
ALTER TABLE sales SET ROW FILTER region_filter ON (region);
```

Easy picture: a row filter is a bouncer checking each row's ticket — rows without a valid ticket for you never make it onto the floor. A column mask is a privacy screen taped over one column — the row is there, but that field is blurred unless you're cleared.

The contrast people get wrong — row vs column control: row-level security removes whole rows (vertical slice by who you are); column masking obscures a field's values while the row stays visible. "Managers see only their department's employees" → row filter. "Everyone sees employees but only HR sees salaries in the clear" → column mask.

Dynamic views (functions in CASE/WHERE)

Mechanism	Restricts
Row filter	Which rows a user sees (a UDF returning TRUE for allowed rows)
Column mask	What a column shows (a UDF returning masked/clear value)
Dynamic view	A view using `is_account_group_member()` / `current_user()` in CASE/WHERE to vary output by user

```
-- dynamic view: mask a column for non-HR, filter rows by region
CREATE VIEW main.hr.emp_secure AS
SELECT id, name,
  CASE WHEN is_account_group_member('hr') THEN ssn ELSE 'XXX-XX-****' END AS ssn
FROM main.hr.employees
WHERE is_account_group_member('all_regions') OR region = current_user_region();
```

Exam tip: both are implemented with UC functions that typically check group membership via `is_account_group_member(...)` or the current user. The policy lives with the table, so it applies no matter how the table is queried.

Subdomain 7.4 — Unity Catalog ABAC policies

> *Understand Unity Catalog ABAC policies to centrally control row-level filtering and column masking for sensitive data.*

Attribute-Based Access Control (ABAC) centralizes row filtering and column masking. Instead of attaching a function to each table one by one, you tag data with governed tags and write one policy that applies wherever that tag appears.

ABAC building block	Role
Governed tag	An attribute attached to columns/tables (e.g. `pii = ssn`, `classification = confidential`)
Policy	A rule (row filter or column mask) attached at catalog/schema/table level, evaluated dynamically
Condition	Targets objects by their tags — the policy fires only where the tag exists

Easy picture: traditional masking is putting a lock on every drawer individually — thousands of locks to install and maintain. ABAC is a rule that says "anything labeled CONFIDENTIAL is automatically locked." You just slap the label on new data and it's protected instantly, everywhere.

The contrast people get wrong — per-table policy vs ABAC: classic column masks / row filters are attached table by table — great for a few tables, unmanageable across thousands. ABAC attaches one policy at a catalog/schema and it protects every tagged column/table underneath, including new tables that get the tag later. Rule: "centrally enforce masking/filtering for all sensitive data across many tables" → ABAC policy + governed tags.

Trap — governed tags & inheritance: ABAC requires governed tags (admin-controlled keys, auditable) — free-form/ungoverned tags are not evaluated by ABAC. And catalog/schema tags inherit to tables for table-level policies, but do not auto-propagate to columns — each column a mask targets must be tagged explicitly.

Exam tip: ABAC (row filtering + column masking policies, governed tags, and automated data classification) is generally available in Unity Catalog. Key value: scale and consistency — tag once, policy applies everywhere and automatically to new tagged data. Recognize it as the centralized evolution of per-table masks/filters.

Worked Examples & Field Notes

Governance is 15% of the exam and almost entirely rule-based, which means it's very learnable. Master three things: the traversal chain, the GRANT/REVOKE/DENY precedence, and managed vs external.

Walkthrough: the "SELECT granted but still denied" mystery

An analyst says "you gave me SELECT but I still can't query the table!" You did nothing wrong — they're missing the traversal grants. To read a table you need all three doors unlocked:

```
GRANT USE CATALOG ON CATALOG main        TO `analysts`;    -- front door
GRANT USE SCHEMA   ON SCHEMA   main.sales TO `analysts`;   -- room door
GRANT SELECT       ON TABLE    main.sales.orders TO `analysts`;   -- the cabinet
```

Easy picture: having SELECT without the USE grants is like being handed the key to a filing cabinet in a building you're not allowed to enter. The key is real. You're still standing outside in the rain.

Walkthrough: the REVOKE that does nothing

Bob is in the reporting group, which has SELECT on a table. Someone runs `REVOKE SELECT ON TABLE ... FROM bob`. What happens when Bob queries? It still works. Why? Because Bob never had a personal grant to remove — his access flows through the group, which nobody touched. REVOKE only cancels a grant made to that exact principal.

```
To actually block Bob, one of:
  DENY SELECT ON TABLE ... TO bob        -- explicit block, beats the group GRANT
  REVOKE SELECT ON TABLE ... FROM reporting     -- remove the group's grant
  remove Bob from the reporting group
```

> **REVOKE vs DENY:** REVOKE removes a specific grant (and does nothing if access comes from elsewhere). DENY is an explicit "no" that overrides any GRANT, including inherited group grants. Precedence to memorize: DENY beats GRANT, always.

Coffee break. *REVOKE-ing a permission the user gets from a group is like taking one candle off a birthday cake and being surprised the room is still lit. The group grant is the other forty candles. Blow those out (DENY) or nothing changes.*

Walkthrough: managed vs external, felt through DROP

```
DROP TABLE main.silver.managed_orders;    -- deletes metadata AND the data files. Gone.
DROP TABLE main.silver.external_orders;   -- deletes ONLY the metadata. Files remain in the bucket.
```

The difference is ownership. A managed table's files live in Unity Catalog's storage, so UC feels free to delete them with the table. An external table's files live at a path you gave it, so UC won't touch them — it only ever managed a pointer.

Real talk. *The moment an engineer learns the difference between managed and external tables is usually the moment right after they dropped a managed table thinking it was external. It's a rite of passage. Try to have yours in a practice workspace, not prod.*

Walkthrough: why ABAC beats a thousand little masks

You could attach a column mask to every table with sensitive data, one at a time, forever, and re-do it for every new table. Or you tag columns `pii` once and write one ABAC policy that protects anything tagged `pii` — including tables that don't exist yet.

> **Exam tip:** "centrally enforce masking/filtering across thousands of tables, including future ones" → ABAC + governed tags, not per-table masks. Tag once, policy applies everywhere.

Coffee break. *Applying column masks table-by-table across a thousand tables is a great plan if your goal is job security through sheer manual toil. ABAC is for people who'd like to also have a weekend.*

One-Minute Recap

Hierarchy: metastore → catalog → schema → object; grants inherit downward; querying a table needs `USE CATALOG` + `USE SCHEMA` + `SELECT`. Managed vs external: DROP on managed deletes data files; DROP on external keeps

files (only metadata removed); external needs an external location + storage credential; `ALTER TABLE … SET MANAGED` converts. Access control: GRANT gives, REVOKE removes, DENY is an explicit block that overrides GRANT (even inherited). Grant to groups, not users. Principals = users, groups, service principals. Volume privileges = READ/WRITE VOLUME (not SELECT/INSERT). Fine-grained: row filter = hide whole rows by who you are; column mask = obscure a field's values (row stays); both are UC functions checking `is_account_group_member(...)`; dynamic views vary output per user. ABAC (GA): tag data with governed tags, write one policy (row filter / column mask) that applies wherever the tag appears, including new tagged tables — the centralized, scalable evolution of per-table masks/filters.

FINAL REVIEW

Master Cheat Sheet, Traps & Exam Strategy

Review these pages the morning of the exam

1 · One-Page Master Cheat Sheet (all 7 sections)

§1 Platform: lakehouse = cheap lake storage + warehouse reliability (Delta) + governance (UC). Control plane (Databricks) vs compute plane (classic=your VMs / serverless=Databricks VMs); data always in your cloud storage. Delta = Parquet + `_delta_log` → ACID, time travel, MERGE=upsert, VACUUM (7-day). Compute: all-purpose (interactive$) · jobs (scheduled, cheap) · SQL warehouse (BI, Photon) · serverless (instant). Cost=DBU×rate+VM. Cluster policy=enforce; pool=fast start.

§2 Ingestion: `spark.read` (finite) vs `readStream` (incremental, needs checkpoint). Auto Loader=`cloudFiles`, real type in `cloudFiles.format`, needs `schemaLocation` + `checkpointLocation`; directory-listing vs file-notification; `addNewColumns` evolution + `_rescued_data`. COPY INTO=SQL idempotent batch. Lakeflow Connect: standard (storage/stream) vs managed (Salesforce/Workday/SQL Server). JDBC=DBs, REST=APIs, both via Jobs.

§3 Transform: methods reshape table (`withColumn` / `distinct` / `dropDuplicates`), functions compute a column (`when` / `row_number`). Joins: inner/left/semi/anti/cross; `broadcast()` small side; Spark union=UNION ALL (keeps dups, by position). Dedup latest+all cols = window+`row_number()==1`. `count("*")` vs `count(col)`; `approx_count_distinct`. Tuning: shuffle.partitions(200), autoBroadcastJoinThreshold(10MB, -1 disables, in-memory not file size); driver=collect, executor=stage. Gold: view(live)/materialized view(stored+REFRESH)/streaming table/table. Quality: CHECK constraint (write fails, table-wide) vs pipeline `expect` / `expect_or_drop` / `expect_or_fail`. MERGE=idempotent.

§4 Jobs: job=DAG of tasks; parallel siblings, fan-in waits for all. Retries=transient only. If/else + for-each in the DAG. Task values pass data. Triggers: scheduled(time) vs file-arrival/table-update(data-driven) vs continuous. Repair run=re-run only failed tasks. Run production on job clusters.

§5 CI/CD: Git Folders (feature branch→PR→merge triggers deploy; code not data). Automation Bundles=`databricks.yml` (resources/targets/variables); dev mode (prefixed, paused) vs prod mode (service principal). CLI: validate→deploy -t→run. Same code, config per target via `-t`. plan=preview, summary=overview; `--var="k=v"`; `type: complex`.

§6 Troubleshoot: run history=trend vs baseline; DAG=find upstream blocker. Spark UI: skew (one task≫others, Max≫Median → AQE skew handling/salt), shuffle (broadcast/filter), spill (memory→disk, slow≠crash). OOM: driver(collect/toPandas) vs executor(skew/partitions). Startup=cloud quota/IAM. Liquid Clustering(CLUSTER BY, changeable keys); predictive optimization(auto OPTIMIZE/VACUUM, serverless).

§7 Governance: query needs USE CATALOG+USE SCHEMA+SELECT. DENY>GRANT; REVOKE only removes a direct grant (group grant survives). Managed drop deletes files; external keeps them (needs external location+credential). Row filter=hide rows; column mask=hide values. ABAC=governed tags + one policy across many/future tables.

2 · Universal Trap List (the distractors that catch people)

If you see...	The trap / right answer
"disable broadcast join"	Set `autoBroadcastJoinThreshold = -1` (a positive value only moves the cutoff)
"keep latest row per key with all columns"	Window + `row_number()==1`, not dropDuplicates (arbitrary) or groupBy (loses cols)
Broadcast eligibility on a Parquet file	Spark measures uncompressed in-memory size, not the compressed file size
OOM on `collect()` / `toPandas()`	Driver memory, not executor
One task 10× slower, Max≫Median	Data skew → AQE skew handling (not bigger cluster, not fewer partitions)
SELECT granted but access denied	Missing `USE CATALOG` / `USE SCHEMA` traversal
REVOKE from a user who's in a granted group	Still has access via the group; need DENY to block
"add a quality rule to a standard Delta table"	CHECK constraint (not a DLT expectation — those need a pipeline)
"track violations but keep the rows"	`expect` (not expect_or_drop/fail)
PySpark `union()` and duplicates	Keeps dups (= UNION ALL); add `.distinct()`
Multi-line / pretty JSON reads as null	`multiLine=true` (default is one-object-per-line)
Auto Loader format option	Outer format=`cloudFiles`; real type in `cloudFiles.format`
Dropped table also deleted files	It was a managed table
"scheduled job runs even with no new data"	Switch to a data-driven trigger (file arrival / table update)
Recover deleted rows fast	RESTORE / time travel, not re-ingest
Enforce/standardize compute configs	Cluster policy (proactive), not a cleanup script (reactive)

3 · Exam-Day Strategy

Do	Why
Read the last sentence of the question first	It states what's actually asked (cheapest? force shuffle? keep columns?)
Eliminate 2 obviously-wrong options, then use a contrast rule	Most questions are two plausible answers + two distractors
Watch for absolute words: "always/never/only"	Often signal a wrong option
Flag-and-return anything >90 seconds	~2 min/question budget; don't sink time early
Never leave a blank	No penalty for guessing — eliminate and pick
Trust the trigger words	"incremental/new files"=Auto Loader; "instant/no infra"=serverless; "enforce config"=cluster policy; "block regardless of group"=DENY

Final 24 hours: re-read the master cheat sheet (§1) and the trap list (§2) twice; redo every "One-Minute Recap" box; skim the contrast callouts (they're the exact decisions the exam tests). Don't cram new topics — consolidate. You've got this.

4 · What This Guide Does & Doesn't Cover (be honest with yourself)

This guide is built from the official May-2026 exam outline and current Databricks documentation, and covers every listed objective across all seven sections with worked examples. It is a study aid, not a question dump. Two honest caveats:

- Exact API option spellings and newest feature names evolve — for anything you're unsure of (Auto Loader options, ABAC DDL, bundle YAML keys), confirm against docs.databricks.com before exam day.
- Nothing substitutes for hands-on practice. Build a pipeline in Databricks Free Edition so the code is muscle memory, not just recognition. The exam rewards people who have actually done the tasks.

One last coffee break. *Remember: the exam is 45 questions, not 45 personal attacks. Most of them are two plausible answers and two obvious distractors wearing a trench coat. Read the last line, eliminate the clowns, apply a contrast rule, and move on. You've read a whole book about this — the exam has read nothing about you.*

You now have all seven sections, deep, in one place: Platform, Ingestion, Transformation, Jobs, CI/CD, Troubleshooting, Governance — plus the blueprint, a master cheat sheet, the universal trap list, and an exam-day plan. Study Sections 3, 2, 4 hardest; treat 7 as high-value quick wins; use the contrast rules to choose between close answers. Now close the guide, build a pipeline, and go pass it. You've got this.

BONUS

Full-Length Mock Exam

45 questions · 90 minutes · single best answer · ~70% (32/45) to pass

This mock mirrors the real blueprint: Platform 3, Ingestion 9, Transformation 9, Jobs 7, CI/CD 5, Troubleshooting 5, Governance 7. Sit it in one 90-minute block with nothing open. Read the last line of each question first, eliminate two, then apply a contrast rule. The answer key with explanations and a scoring guide starts after Q45 — no peeking until you're done.

> **How to score yourself.** Mark each answer, then flip to the key. Count correct out of 45. 32+ = on track (~70%). Below that, note which domains you missed most and send those hours back into the matching chapter's One-Minute Recap and contrast boxes. A wrong answer you understand afterward is worth more than a lucky guess.

DOMAIN 1 · DATABRICKS INTELLIGENCE PLATFORM (Q1–Q3)

Q1. A security reviewer asks where the rows of a Delta table registered in Unity Catalog physically live after a notebook query runs. What is the correct answer?

A. In the Databricks control plane alongside the Unity Catalog metadata.
B. In your own cloud object storage (S3 / ADLS / GCS).
C. On the local disk of the driver node in the compute plane.
D. In a proprietary Databricks-managed database.

Q2. A nightly ETL pipeline must "just run" on a schedule as cheaply as possible, with no interactive users. Which compute should it target?

A. An all-purpose cluster with auto-termination.
B. A SQL warehouse.
C. Jobs compute (a job cluster).
D. A larger all-purpose cluster to finish faster.

Q3. An admin wants to stop users from ever creating oversized clusters or clusters without auto-termination. What enforces this at creation time?

A. A scheduled script that terminates expensive clusters.
B. A cluster policy.
C. A cluster pool.
D. Turning on autoscaling.

DOMAIN 2 · DATA INGESTION AND LOADING (Q4–Q12)

Q4. Millions of small files land continuously in a cloud bucket and the schema occasionally gains new columns. Which ingestion tool fits best?

A. A scheduled `spark.read` of the whole directory.
B. COPY INTO on a cron schedule.
C. Auto Loader (`cloudFiles`).
D. A Lakeflow Connect managed connector.

Q5. You are configuring Auto Loader to read JSON. Which option assignment is correct?

A. `.format("json")` and `.option("cloudFiles.format","cloudFiles")`
B. `.format("cloudFiles")` and `.option("cloudFiles.format","json")`
C. `.format("autoloader")` and `.option("format","json")`
D. `.format("cloudFiles")` and `.option("format","json")`

Q6. In a robust Auto Loader stream, what is the role of `cloudFiles.schemaLocation` as distinct from `checkpointLocation`?

A. schemaLocation stores streaming progress; checkpointLocation stores the schema.

B. Both store the same thing and can point to one folder.

C. schemaLocation stores the inferred schema (so evolution is detected); checkpointLocation stores which files are already processed.

D. schemaLocation is optional for streams and only used in batch reads.

Q7. A COPY INTO job is scheduled to run every night against the same landing folder. On the second run, no new files have arrived. What happens?

A. It reloads every file, duplicating the data.

B. It errors because the files were already loaded.

C. It loads nothing new — it tracks loaded files and skips them (idempotent).

D. It only works once and must be recreated each night.

Q8. Which COPY INTO setting will cause already-loaded files to be re-ingested, producing duplicate rows?

A. `COPY_OPTIONS('mergeSchema'='true')`

B. `FORMAT_OPTIONS('inferSchema'='true')`

C. `COPY_OPTIONS('force'='true')`

D. Setting `FILEFORMAT = PARQUET`

Q9. The team needs to pull Salesforce and Workday data into Unity Catalog with little or no code and built-in governance. Which approach fits?

A. Auto Loader pointed at an S3 export bucket.

B. A Lakeflow Connect managed connector.

C. A JDBC read in a notebook.

D. COPY INTO from the SaaS API.

Q10. A streaming write is started without a `checkpointLocation`. What is the consequence?

A. Nothing — checkpoints are optional for streams.

B. There is no exactly-once guarantee; on restart it cannot resume and would re-process data.

C. The schema can no longer evolve.

D. The stream is automatically converted to a batch read.

Q11. A pretty-printed JSON file (one object spanning many lines) is read and every row comes back null or corrupt. What fixes it?

A. `.option("multiLine","true")`

B. `.option("inferSchema","true")`

C. `.option("mode","FAILFAST")`

D. `.option("header","true")`

Q12. In Databricks SQL, a column `raw_json` holds a JSON string. Which is a valid way to extract the nested field `user.id`?

A. `extract_json(raw_json,'user.id')`

B. `raw_json:user.id` (colon operator) or `get_json_object(raw_json,'$.user.id')`

C. `raw_json->>'user.id'`

D. `json_parse(raw_json).user.id`

DOMAIN 3 · DATA TRANSFORMATION AND MODELING (Q13–Q21)

Q13. You must keep only the latest row per customer key, retaining all columns, deterministically. Which approach is correct?

A. `dropDuplicates(["key"])`

B. `groupBy("key").max(...)`

C. A window with `row_number()` ordered by timestamp desc, keep `rn == 1`.

D. `distinct()`

Q14. A query joins one huge fact table to a very small dimension table. What is the recommended optimization?

A. Increase `spark.sql.shuffle.partitions`.

B. `broadcast()` the small side.

C. Repartition the fact table by the join key.

D. Cache both tables first.

Q15. Two DataFrames are combined with PySpark `df1.union(df2)`. What is true of the result?

A. Duplicates are removed automatically.

B. It matches columns by name.

C. It keeps duplicates (like UNION ALL) and matches by position; add `.distinct()` to dedup.

D. It fails unless both schemas are identical objects.

Q16. You must expand an array column into one row per element but must keep rows whose array is empty or null. Which function?

A. `explode()`

B. `explode_outer()`

C. `flatten()`

D. `posexplode()`

Q17. You need a distinct-count of billions of rows where a small margin of error is acceptable, as fast as possible. Which function?

A. `countDistinct(col)`

B. `count("*")`

C. `approx_count_distinct(col)`

D. `distinct().count()`

Q18. You must reject writes that violate a rule on a standard Delta table (not a pipeline). Which mechanism?

A. A CHECK constraint.

B. A DLT `CONSTRAINT ... EXPECT`.

C. A bare `EXPECT` expectation.

D. A row filter function.

Q19. A Gold-layer object serves a BI aggregation/join and its source table receives updates and deletes. Which object recomputes correctly and refreshes incrementally?

A. A streaming table (append-only).

B. A plain view.

C. A materialized view.

D. A global temp view.

Q20. A daily load might occasionally run twice. Which write pattern is safe to re-run without creating duplicates?

A. append mode on the business key.

B. MERGE on the business key.

C. overwrite the whole target every run.

D. insertInto with dynamic partitions.

Q21. You want to completely disable automatic broadcast joins. What do you set `spark.sql.autoBroadcastJoinThreshold` to?

A. 0
B. A very large positive number.
C. -1
D. 10MB

DOMAIN 4 · WORKING WITH LAKEFLOW JOBS (Q22–Q28)

Q22. In a job DAG, tasks B and C both depend only on task A and have no link to each other. How do B and C run?

A. Sequentially, in the order defined.
B. Concurrently, after A completes.
C. Only if a condition task allows it.
D. C waits for B to finish.

Q23. A task intermittently fails due to a momentary network blip. What is the appropriate control-flow feature, and what will it not fix?

A. Task retries; they will not fix a deterministic code bug.
B. A bigger cluster; it fixes all failures.
C. A condition task; it retries automatically.
D. Continuous mode; it prevents all failures.

Q24. A job should start as soon as new files land in a volume or external location. Which trigger type?

A. A cron schedule every minute.
B. File arrival trigger.
C. Continuous mode.
D. Manual trigger.

Q25. A scheduled job keeps firing on the clock even when no new data has arrived, wasting compute on empty runs. What is the fix?

A. Increase the schedule frequency.
B. Switch to a data-driven trigger (file arrival or table update).
C. Add more retries.
D. Move to a larger cluster.

Q26. A 6-task job failed at task 5. You fixed the bug. What is the cheapest way to complete the run?

A. Re-run the entire job from task 1.
B. Use Repair run to re-execute only the failed/remaining tasks.
C. Clone the job and run the clone.
D. Delete and recreate the job.

Q27. You want a notification/cleanup task to run even if an upstream task failed. What run-if / dependency condition do you set?

A. All succeeded.
B. All done (run regardless of upstream success/failure).
C. None failed.
D. At least one succeeded, only.

Q28. Following best practice, production job tasks should run on which compute and as which identity?

A. An all-purpose cluster, as the developer's user account.
B. A job cluster, as a service principal.

C. A SQL warehouse, as a group.
D. Serverless, as an admin user.

DOMAIN 5 · IMPLEMENTING CI/CD (Q29–Q33)

Q29. You want to preview what a Databricks Asset Bundle deployment would change, without actually deploying. Which command?

A. `databricks bundle show`
B. `databricks bundle plan`
C. `databricks bundle describe`
D. `databricks bundle deploy --dry`

Q30. In a CI pipeline you want to check the bundle configuration is valid before any deploy step. Which command is the gate?

A. `databricks bundle validate`
B. `databricks bundle summary`
C. `databricks bundle run`
D. `databricks bundle destroy`

Q31. You need to override a bundle variable at deploy time from the CLI. Which syntax is correct?

A. `-v key:value`
B. `--var="key=value"`
C. `--set key value`
D. `-var key=value`

Q32. You must promote the same code to dev, test, and prod, changing only configuration. How is this done with bundles?

A. Maintain three separate copies of the code.
B. One bundle, deploy with `-t <target>` to swap environment config.
C. A global cluster policy per environment.
D. Manually reconfigure jobs in each workspace UI.

Q33. Which Git workflow correctly triggers an automated deployment in a CI/CD setup?

A. Commit straight to main on every change.
B. Work on a feature branch, open a PR, and let the merge to main trigger the deploy.
C. Rebase main onto your feature branch before every commit.
D. Push data files through Git alongside the code.

DOMAIN 6 · TROUBLESHOOTING, MONITORING & OPTIMIZATION (Q34–Q38)

Q34. In the Spark UI, one task in a stage runs far longer than the rest and its Max shuffle read is far above the median. What is the cause and first fix?

A. Under-provisioned cluster → add nodes.
B. Too many partitions → coalesce.
C. Data skew → enable AQE skew join handling.
D. Slow disk → switch instance type.

Q35. A job throws an OutOfMemory error during a `collect()` / `toPandas()` call. Which memory ran out?

A. Executor memory.
B. Driver memory.
C. Shuffle-service memory.
D. Cloud storage quota.

Q36. The Spark UI shows spill (disk) for a stage, but the job finished successfully. How should you interpret this?

A. The job actually failed and must be re-run.
B. Memory overflowed to disk — slower, but not a failure (unlike an OOM crash).
C. Data was silently dropped.
D. The driver crashed and recovered.

Q37. A cluster won't start, citing a capacity/quota problem. Where is the root cause, and where do you look?

A. In your notebook code — check the stack trace.
B. Cloud-side quota / IAM — check the cluster event log.
C. Unity Catalog permissions — check GRANTs.
D. The Spark UI — check the DAG.

Q38. You want tables kept optimized (compaction, cleanup) automatically, without scheduling your own maintenance jobs. What do you use?

A. Manual OPTIMIZE + VACUUM on a cron.
B. Predictive optimization.
C. A larger cluster.
D. ZORDER on every write.

DOMAIN 7 · GOVERNANCE AND SECURITY (Q39–Q45)

Q39. A table was dropped and its underlying data files were also physically deleted. What kind of table was it?

A. An external table.
B. A managed table.
C. A view.
D. A foreign table.

Q40. A user has been granted SELECT on a table but still gets "permission denied." What is missing?

A. MODIFY on the table.
B. USE CATALOG on the catalog and USE SCHEMA on the schema.
C. Ownership of the table.
D. A row filter exemption.

Q41. You REVOKE SELECT from a user, but they still access the table because they belong to a group that was granted SELECT. How do you block them?

A. REVOKE again.
B. Apply DENY to the user (DENY overrides GRANT).
C. Drop the user from Unity Catalog.
D. Add a second GRANT with lower priority.

Q42. You must hide entire rows from certain users based on who they are (not blank out a field). Which control?

A. A column mask.
B. A row filter (row-level security).
C. A CHECK constraint.
D. Table ownership transfer.

Q43. You need to enforce masking consistently across thousands of existing tables and any created in the future. What is the right tool?

A. A per-table column mask on each table.
B. An ABAC policy with governed tags.

C. A DENY on each table.
D. A cluster policy.

Q44. You want to create an external table over an existing S3 path. What two objects must exist?

A. A cluster policy and a pool.
B. An external location and a storage credential.
C. A volume and a checkpoint.
D. A metastore and a warehouse.

Q45. A service principal must be able to upload files into a Unity Catalog volume. Which privilege do you grant?

A. INSERT
B. WRITE VOLUME
C. MODIFY
D. READ VOLUME

Answers & Explanations

Score 32+/45 (~70%) to be on track · note which domains you miss

Quick key. 1-B · 2-C · 3-B · 4-C · 5-B · 6-C · 7-C · 8-C · 9-B · 10-B · 11-A · 12-B · 13-C · 14-B · 15-C · 16-B · 17-C · 18-A · 19-C · 20-B · 21-C · 22-B · 23-A · 24-B · 25-B · 26-B · 27-B · 28-B · 29-B · 30-A · 31-B · 32-B · 33-B · 34-C · 35-B · 36-B · 37-B · 38-B · 39-B · 40-B · 41-B · 42-B · 43-B · 44-B · 45-B

Q1 — B. Your data always stays in your cloud object storage. The control plane holds only metadata/config; the compute plane executes but doesn't store the data.

Q2 — C. Jobs compute has a cheaper DBU rate and terminates after the run. All-purpose is ~2–3× the rate and stays up idle; bigger isn't cheaper.

Q3 — B. A cluster policy prevents bad configs at creation (proactive). A kill script is reactive — money's already spent. Pools speed startup; autoscaling doesn't restrict configs.

Q4 — C. Auto Loader scales to millions of files, tracks state, and evolves schema. COPY INTO suits smaller periodic SQL batches; a plain read reloads everything.

Q5 — B. The outer format is always `cloudFiles`; the real file type goes in `cloudFiles.format`. That inversion is a classic trap.

Q6 — C. Two folders, two jobs: schemaLocation persists the inferred schema so evolution is detectable; checkpointLocation tracks processed files. Both are needed.

Q7 — C. COPY INTO is idempotent: it records loaded files in the transaction log and skips them. Re-running with no new files loads nothing.

Q8 — C. `force=true` bypasses file tracking and re-ingests everything, duplicating rows. Default (no force) is the safe idempotent behavior.

Q9 — B. SaaS apps/databases → Lakeflow Connect managed connector (UI-driven, governed, little code). Auto Loader is for files in buckets; JDBC is code-based fallback.

Q10 — B. The checkpoint stores processed offsets/files; without it there's no exactly-once guarantee and a restart can't resume. Checkpoints are mandatory for streams.

Q11 — A. Spark defaults to newline-delimited JSON (JSONL). A pretty-printed or array file needs `multiLine=true`.

Q12 — B. Databricks SQL uses the colon operator (`col:field.sub`) or `get_json_object`. `extract_json()` is the invented distractor.

Q13 — C. Window + `row_number()` desc, keep `rn==1`, keeps all columns deterministically. dropDuplicates keeps an arbitrary row; groupBy+max loses columns.

Q14 — B. Broadcast the small side to avoid shuffling the huge one. Spark measures uncompressed in-memory size, not file size.

Q15 — C. PySpark `union()` = UNION ALL: keeps duplicates and matches by position. Add `.distinct()` to dedup, or use `unionByName` to match by name.

Q16 — B. `explode_outer` keeps rows with empty/null arrays; plain `explode` drops them.

Q17 — C. `approx_count_distinct` is fast with a small error. Exact `countDistinct` must shuffle and is slower.

Q18 — A. Standard Delta table → CHECK constraint (rejects bad writes for everyone). EXPECT expectations only work inside a DLT/Lakeflow pipeline.

Q19 — C. A materialized view stores results and refreshes incrementally, recomputing joins over updated/deleted sources. Streaming tables are append-only; plain views recompute every query.

Q20 — B. MERGE on the business key is idempotent — a re-run updates in place instead of duplicating. A plain append duplicates.

Q21 — C. -1 disables broadcast joins entirely. A positive value only moves the size cutoff; 0 doesn't fully disable it.

Q22 — B. Sibling tasks sharing one parent and unlinked to each other run concurrently once the parent completes. Fan-in tasks wait for all parents.

Q23 — A. Retries handle transient/flaky failures (network blips). They won't fix a deterministic code bug — it'll just fail every retry.

Q24 — B. File arrival trigger reacts to files landing in a volume/external location. A table-update trigger reacts to Delta commits.

Q25 — B. Empty runs mean a clock-based schedule fires with no data. Switch to a data-driven trigger (file arrival / table update) so it only runs when there's work.

Q26 — B. Repair run re-executes only the failed/remaining tasks — cheapest after a fix. Re-running all 6 wastes the successful work.

Q27 — B. "All done" runs the task regardless of upstream success or failure — right for cleanup/notification tasks. "All succeeded" would skip it on failure.

Q28 — B. Production runs on a job cluster (cheaper, ephemeral) as a service principal, so pipelines don't break when a person leaves.

Q29 — B. `databricks bundle plan` previews changes without deploying. There is no `bundle show` / `describe`.

Q30 — A. `databricks bundle validate` checks config before deploy — the CI gate. `summary` is a human-readable overview, not a validation gate.

Q31 — B. `--var="key=value"` — double-dash, equals, quotes. Not `-v`, not a colon.

Q32 — B. One bundle, swap environments with `-t <target>`. Same code, config per target — the golden rule of this domain.

Q33 — B. Feature branch → PR → merge to main triggers deploy. Never commit straight to main; rebase rewrites history and is discouraged; Git versions code, not data.

Q34 — C. One task ⩾ others with Max shuffle read ⩾ median is the signature of data skew. First-line fix is AQE skew join handling — not a bigger cluster or fewer partitions.

Q35 — B. `collect()` / `toPandas()` pull data to the driver, so OOM there is driver memory. Executor OOM happens mid-shuffle/skew.

Q36 — B. Spill = memory overflowed to disk: slow, not a crash. An OOM is the crash. A finished job that spilled succeeded, just inefficiently.

Q37 — B. Capacity/quota failures are cloud-side (quota/IAM), surfaced in the cluster event log — not in your code or the Spark UI.

Q38 — B. Predictive optimization runs OPTIMIZE/VACUUM automatically on managed serverless. Use Liquid Clustering when you need to change layout keys later.

Q39 — B. Dropping a managed table deletes its files (UC owns the storage). External tables keep their files — only the metadata is removed.

Q40 — B. SELECT never grants traversal. The user also needs `USE CATALOG` and `USE SCHEMA` to reach the object.

Q41 — B. The group grant still applies after revoking the user's direct grant. DENY overrides GRANT and blocks the user.

Q42 — B. A row filter hides whole rows by identity; a column mask hides a field's value.

Q43 — B. ABAC policies + governed tags apply one masking rule across many tables and future ones. Per-table masks don't scale and miss new tables.

Q44 — B. External tables/volumes on a cloud path need an external location backed by a storage credential.

Q45 — B. Volumes hold files, so privileges are file privileges: WRITE VOLUME to upload, READ VOLUME to read. INSERT/SELECT are for table rows, not volumes.

> **After you score.** Tally misses per domain. If you dropped 2+ in any single domain, that's your next study block — go straight to that chapter's One-Minute Recap and its "contrast people get wrong" boxes, which are the exact decisions these questions test. Re-sit this mock in a few days; aim to clear 38/45 before exam day so a 70% cut score has comfortable margin.